
Shrinking the Variance: Shrinkage Baselines for Reinforcement Learning with Verifiable Rewards

Guanning Zeng¹ Zhaoyi Zhou^{1†} Daman Arora¹ Andrea Zanette¹

Abstract

Reinforcement Learning with Verifiable Rewards (RLVR) has emerged as a powerful paradigm for post-training large reasoning models (LRMs) using policy-gradient methods such as GRPO. To stabilize training, these methods typically center trajectory rewards by subtracting the empirical mean for each prompt. Statistically, this centering acts as a *control variate* (or *baseline*), reducing the variance of the policy-gradient estimator. Typically, the mean reward is estimated using per-prompt empirical averages for each prompt in a batch. Drawing inspiration from Stein’s paradox, we propose using *shrinkage estimators* that combine *per-prompt* and *cross-prompt* means to improve the overall per-prompt mean estimation accuracy, particularly in the low-generation regime typical of RLVR. Theoretically, we construct a shrinkage-based baseline that provably yields lower-variance policy-gradient estimators across algorithms. This baseline serves as a drop-in replacement for existing per-prompt mean baselines, requiring no additional hyperparameters or computation. Empirically, shrinkage baselines consistently outperform standard empirical-mean baselines, leading to lower-variance gradient updates and improved training stability.

1. Introduction

Large reasoning models such as *OpenAI-o1* (OpenAI, 2024) and *DeepSeek-R1* (Guo et al., 2025) have demonstrated impressive reasoning capabilities, underscoring the effectiveness of reinforcement learning (RL) techniques for model post-training. A particularly impactful paradigm for fine-tuning reasoning models is **Reinforcement Learning with**

Verifiable Rewards (RLVR), where models are optimized using sparse, rule-based scalar rewards explicitly indicating the correctness of the model’s final answer. RLVR-style training has shown substantial promise for tasks that require explicit and verifiable logic, such as mathematical or logical reasoning.

A common approach to RLVR is to apply policy gradient methods such as REINFORCE (Williams, 1992) or more modern variants such as GRPO (Shao et al., 2024), where the policy is optimized to maximize expected rewards through stochastic gradient estimates. A well-known challenge in policy gradient methods is the high variance of these gradient estimators (Sutton & Barto, 2018), which can hinder stable training. To mitigate this, RL methods introduce a baseline (known in statistics as a control variate), which is a state-dependent shift in the rewards that reduces variance in the gradient without introducing bias (Sutton et al., 1998; Greensmith et al., 2004). The canonical choice is the value function, defined as the expected return from the state under consideration (or the initial state in RLVR) under the current policy. Despite being a heuristic rather than the theoretically optimal baseline, it is widely adopted (Sutton & Barto, 2018) because it is straightforward to approximate while providing substantial variance reduction.

Typically, the value function itself is unknown and it must be estimated. There are two broad classes of value function estimators. *Classical RL approaches* introduce an auxiliary neural network to approximate the value function (Barto et al., 1989; Mnih et al., 2016; Haarnoja et al., 2018; Schulman et al., 2015; 2017). When carefully tuned, this strategy can be effective, but it comes with significant practical challenges: increased hyperparameter sensitivity, added engineering complexity, and the cost of training and maintaining an additional network for variance reduction. In contrast, *recent methods for reasoning models*, including GRPO (Shao et al., 2024), RLOO (Ahmadian et al., 2024), ReMax (Li et al., 2024), REINFORCE++ (Hu et al., 2025), DAPO (Yu et al., 2025) and CISPO (MiniMax, 2025), forgo explicit value function approximation. Instead, they construct baselines directly from Monte Carlo returns, most commonly using per-prompt empirical averages of generated responses. This avoids the overhead of a separate network and yields

[†]Led the theoretical development. ¹Carnegie Mellon University. Correspondence to: Andrea Zanette <zanette@cmu.edu>, Guanning Zeng <guanninz@andrew.cmu.edu>.

unbiased (or nearly unbiased) estimates, making it attractive for large reasoning models.

However, compared to classical domains, RL training on LLMs typically operates with large batch sizes but *fewer rollouts per prompt* due to the high cost of inference. Even in industrial-scale LLM training (Guo et al., 2025; DeepSeek-AI et al., 2025), the number of rollouts per prompt is often limited to 16 or fewer. This presents a new challenge: accurately estimating the baseline itself becomes difficult when the number of samples is small.

In this work, we revisit this problem of estimating value functions in RLVR. Although the per-prompt sample mean is an unbiased estimator, it treats each prompt independently. By recognizing that value functions must be estimated **simultaneously** across all prompts in a batch, we can construct an estimator for the value function (and thus the gradient) with strictly lower mean squared error (MSE). In particular, we propose a new baseline estimator inspired by the classical James–Stein shrinkage principle (James et al., 1961; Stein et al., 1956). This estimator reduces variance by trading a small amount of bias in the baseline for improved overall efficiency. Crucially, despite using a biased baseline, the **resulting policy gradient estimator remains unbiased** and enjoys provable variance reduction under standard assumptions.

Our proposed baseline introduces **no additional hyperparameters**, making it a simple drop-in replacement for existing critic-free RL methods. It relies solely on frequentist principles, without requiring assumptions about task difficulty, training data distributions, or model architectures. Importantly, the shrinkage baseline can be computed with **negligible computational overhead**. Extensive experiments across diverse models, tasks, and rollout settings demonstrate that the shrinkage baseline estimator consistently outperforms other common baselines in variance reduction. Furthermore, we observe a significant decrease in policy gradient variance, aligning with our theoretical predictions.

2. Preliminaries

Let π_θ denote the language model parameterized by θ . Given a prompt x from distribution $\mathcal{D}$, the model outputs response $y \sim \pi_\theta(\cdot | x)$ and receives a deterministic, verifiable reward $r(x, y)$ (e.g., correctness in math tasks). The RL objective is to maximize the expected reward

$$J(\theta) := \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)}[r(x, y)].$$

The vanilla REINFORCE algorithm (Williams, 1992) derives the policy gradient as

$$\nabla_\theta J(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)}[r(x, y) \nabla_\theta \log \pi_\theta(y | x)],$$

so that the gradient can be estimated with one online sample:

$$g^{\text{vanilla}}(x, y; \theta) = r(x, y) \nabla_\theta \log \pi_\theta(y | x).$$

A scalar, prompt-dependent baseline $b(x) \in \mathbb{R}$ can be added to further reduce the gradient variance while keeping the gradient unbiased (Sutton et al., 1998):

$$g^{\text{baseline}}(x, y; \theta) := (r(x, y) - b(x)) \nabla_\theta \log \pi_\theta(y | x).$$

More samples further reduce variance. Thus, at each RL step, it is common to sample n prompts $\mathbf{x} = (x_1, \dots, x_n)$ i.i.d. from $\mathcal{D}$. For each prompt x_i , the model generates m responses $\mathbf{y}_i = (y_i^1, \dots, y_i^m)$ from $\pi_\theta(\cdot | x_i)$ with rewards $r_i^j := r(x_i, y_i^j)$. Let $\mathbf{Y} = (\mathbf{y}_1, \dots, \mathbf{y}_n)$. The policy gradient is estimated by

$$g(\mathbf{x}, \mathbf{Y}; \theta) := \frac{1}{nm} \sum_{i,j} (r_i^j - b_i^j) \nabla_\theta \log \pi_\theta(y_i^j | x_i). \quad (1)$$

Here b_i^j is the baseline for sample (x_i, y_i^j) . Although baselines are typically prompt-dependent (i.e., b_i^j depends only on x_i), we allow per-reward baselines to accommodate leave-one-out estimators like RLOO (Ahmadian et al., 2024).¹

Another requirement for baselines is unbiasedness. As shown in Proposition 1, Equation (1) is an unbiased estimate of the policy gradient $\nabla_\theta J(\theta)$, as long as b_i^j and r_i^j are independent for all $1 \leq i \leq n, 1 \leq j \leq m$.

Proposition 1 (Unbiasedness). *Suppose b_i^j is independent of y_i^j for all i, j . Then $g(\mathbf{x}, \mathbf{Y}; \theta)$ is unbiased, i.e., $\mathbb{E}[g(\mathbf{x}, \mathbf{Y}; \theta)] = \nabla_\theta J(\theta)$.*

We defer the proof to Section A.1. REINFORCE with baseline is a special form of Equation (1) when $n = m = 1$.

Beyond this basic formulation, several practical algorithms have been developed to improve training stability. PPO (Proximal Policy Optimization) (Schulman et al., 2017) is widely used in RLHF pipelines for language models due to its clipping objective, which prevents overly large policy updates. It is defined by the following updates, where we leave the advantage estimator A_t to be specified.

$$\begin{aligned} \mathcal{J}_{\text{PPO}}(\theta) &= \frac{1}{nm} \sum_{i,j} \frac{1}{|y_i^j|} \sum_t \min(\rho_t A_t, \text{clip}(\rho_t, 1 \pm \epsilon) A_t), \\ \rho_t &= \frac{\pi_\theta(y_{i,t}^j | x_i, y_{i,<t}^j)}{\pi_{\theta_{\text{old}}}(y_{i,t}^j | x_i, y_{i,<t}^j)}. \end{aligned}$$

For LLM training, GRPO (Shao et al., 2024) proposes a Z-normalized advantage, where μ_i and σ_i are the prompt-level

¹RLOO computes each baseline by averaging the other rewards for the same prompt except the one being estimated, ensuring independence between b_i^j and r_i^j for unbiasedness.

mean and standard deviation of rewards, and $\delta > 0$ avoids division by zero:

$$A_{i,j}^{\text{GRPO}} = (r_i^j - \mu_i) / (\sigma_i + \delta),$$

$$\mu_i = \frac{1}{m} \sum_{j=1}^m r_i^j, \quad \sigma_i = \sqrt{\frac{1}{m} \sum_{j=1}^m (r_i^j - \mu_i)^2}.$$

More recent work has shown that dividing by σ_i biases the objective (Liu et al., 2025) without improving empirical performance, and can be omitted (DeepSeek-AI et al., 2025; Khatri et al., 2026).

3. Derivation of the Method

3.1. From Policy Gradient Variance to Value Function Estimators

The purpose of a reinforcement learning baseline is to act as a control variate and thereby reduce the variance of the policy gradient estimator. The baseline b should be chosen so as to minimize the variance of the gradient estimator in Equation (1). Since $g(\mathbf{x}, \mathbf{Y}; \theta)$ is vector-valued, its variance is naturally represented by the covariance matrix $\text{Var}[g(\mathbf{x}, \mathbf{Y}; \theta)]$. A common scalar summary is the trace of this matrix, i.e., the sum of coordinate-wise variances, which is equivalent to the mean squared error of the estimator:

$$\text{Var}[g] := \text{Tr}(\text{Var}[g(\mathbf{x}, \mathbf{Y}; \theta)])$$

$$= \mathbb{E}[\|g(\mathbf{x}, \mathbf{Y}; \theta) - \nabla_{\theta} J(\theta)\|_2^2]. \quad (2)$$

In general, the variance-minimizing baseline for a given prompt x depends on the squared norm of the score function $\nabla_{\theta} \log \pi_{\theta}(y | x)$, which is typically expensive to compute (Greensmith et al., 2004). For this reason, a standard simplification in the literature (Sutton et al., 1999; Schulman et al., 2016), often made implicitly, is to ignore the dependence on the score function and directly find a baseline function $b(x)$ that minimizes the mean squared error of the baseline estimator with respect to the observed rewards. In other words, theoretically one would choose a baseline b to minimize the population-level mean squared error for every state x :

$$\mu(x) = \arg \min_b \mathbb{E}[(r(x, y) - b(x))^2]. \quad (3)$$

The optimal baseline is the value function, i.e., the *Bayes-optimal predictor* of the reward under the current policy:

$$\mu(x) := \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)}[r(x, y)], \quad \forall x.$$

Using the value function as the baseline is theoretically justified: it coincides with the variance-minimizing baseline when the squared score-function norms are approximately homogeneous across actions, and we verify the resulting gradient-variance reduction empirically in Section 4 and Section B. The value-function baseline has become the standard foundation for variance reduction in reinforcement learning (Sutton & Barto, 2018), serving as the

core theoretical basis for both classical actor-critic methods (e.g., A3C (Mnih et al., 2016), SAC (Haarnoja et al., 2018), TRPO (Schulman et al., 2015), PPO (Schulman et al., 2017)) and more recent critic-free methods for reasoning models (e.g., ReMax (Li et al., 2024), RLOO (Ahmadian et al., 2024), GRPO (Shao et al., 2024), REINFORCE++ (Hu et al., 2025)).

In practice, the value function is unknown. Simple algebra shows that minimizing Equation (3) leads to **minimizing the mean squared error of the baseline $b(x)$ with respect to the value function $\mu(x)$** , which is the starting point for our development to follow.

$$\mathbb{E}[(r(x, y) - b(x))^2] = \mathbb{E}[(r(x, y) - \mu(x) + \mu(x) - b(x))^2]$$

$$= \mathbb{E}[(r(x, y) - \mu(x))^2] - 2\mathbb{E}[(r(x, y) - \mu(x))(b(x) - \mu(x))]$$

$$+ (b(x) - \mu(x))^2$$

$$= \text{Var}[r(x, y)] + (b(x) - \mu(x))^2. \quad (4)$$

3.2. The Bias-Variance Trade-off for Baselines in RLVR

We then analyze a single RL step with a batch of prompts. Corresponding to Equation (1), policy-gradient updates also require value-function estimates for a *batch* of prompts. This means that the empirical optimization problem associated with Equation (4) is a batch-level mean squared error:

$$\frac{1}{mn} \sum_{i=1}^n \sum_{j=1}^m \mathbb{E}_{\mathbf{Y}}[(b_i^j - \mu_i)^2]. \quad (5)$$

There is a standard bias-variance decomposition of the baseline mean squared error.

$$\frac{1}{mn} \sum_{i=1}^n \sum_{j=1}^m \mathbb{E}[(b_i^j - \mu_i)^2] \quad (6)$$

$$= \frac{1}{mn} \sum_{i,j} \mathbb{E}[(b_i^j - \mathbb{E}[b_i^j] + \mathbb{E}[b_i^j] - \mu_i)^2]$$

$$= \frac{1}{mn} \sum_{i,j} (\mathbb{E}[(b_i^j - \mathbb{E}[b_i^j])^2] + 2\mathbb{E}[(b_i^j - \mathbb{E}[b_i^j])(\mathbb{E}[b_i^j] - \mu_i)]$$

$$+ (\mathbb{E}[b_i^j] - \mu_i)^2)$$

$$= \underbrace{\frac{1}{mn} \sum_{i,j} \mathbb{E}[(b_i^j - \mathbb{E}[b_i^j])^2]}_{\text{Variance}} + \underbrace{\frac{1}{mn} \sum_{i,j} (\mathbb{E}[b_i^j] - \mu_i)^2}_{\text{Bias}^2}.$$

Here expectations are taken over the responses $\mathbf{Y}$, and $\mu_i = \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x_i)}[r(x_i, y)]$ denotes the true value function for prompt x_i . For exposition, we temporarily relax the unbiased-gradient requirement that b_i^j be independent of y_i^j .

The decomposition above highlights a bias-variance trade-off, with different baselines exhibiting distinct behaviors:

- **Prompt-level mean reward**, the most commonly used baseline in current RLVR, e.g., in RLOO (Ahmadian et al., 2024) and GRPO (Guo et al., 2025):

$$\hat{\mu}_i = \frac{1}{m} \sum_{j=1}^m r_i^j. \quad (7)$$

This choice minimizes the bias term, but does not efficiently reduce variance because it ignores informative cross-prompt structure.

- **Global batch mean reward**, as in REINFORCE++ (Hu et al., 2025) and SPO (Xu & Ding, 2026):

$$\hat{\bar{\mu}} = \frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m r_i^j. \quad (8)$$

This baseline often achieves low variance but incurs substantial bias when prompt-specific means differ.

Given the limitations of both baselines, it is natural to consider combining them. While such a combination has not been studied in reinforcement learning of LLMs, a similar question exists in classical statistics. James et al. (1961) and Stein et al. (1956) showed that when jointly estimating three or more means, the empirical mean is suboptimal, and pooling information across tasks strictly reduces total MSE. Although the reward distribution in RLVR is not Gaussian and classical James—Stein guarantees do not directly apply, the principle of borrowing strength across related estimation problems is appealing. This motivates shrinking each prompt-level mean toward the global batch mean, yielding a *shrinkage baseline* in RLVR:

$$b_i^{j, \text{JS1}} = (1 - \lambda) \hat{\mu}_i + \lambda \hat{\bar{\mu}}. \quad (9)$$

The shrinkage coefficient $\lambda \in [0, 1]$ balances variance reduction and bias. The optimal λ admits a closed-form estimate from data (proof in Appendix A.2):

Proposition 2. Let $v = \frac{1}{nm} \sum_{i=1}^n \sigma_i^2$, where $\sigma_i^2 = \text{Var}[r(x_i, y)]$. Let $\bar{\mu} = \frac{1}{n} \sum_{i=1}^n \mu_i$ and $s = \frac{1}{n-1} \sum_{i=1}^n (\mu_i - \bar{\mu})^2$. Then, within the class of linear shrinkage baselines of the form Equation (9), the minimizer of the relaxed baseline-MSE objective in Equation (5) is

$$\lambda^* = \frac{v}{s+v}.$$

Here, v captures the average per-prompt variance, while s measures the dispersion of true value functions across prompts. When rewards of prompts are similar (small s), stronger shrinkage is preferred; when rewards of prompts are heterogeneous (large s), the estimator leans more on local means. **Since $0 < \lambda^* < 1$ whenever $v > 0$ and $s > 0$, within the class of linear shrinkage baselines of the form Equation (9), both the prompt-level mean reward in Equation (7) (corresponding to $\lambda = 0$) and the global batch mean reward in Equation (8) (corresponding to $\lambda = 1$) are strictly dominated under the**

baseline-MSE objective in Equation (5). The interpolated baseline Equation (9) with $\lambda = \lambda^*$ thus achieves the optimal balance between *batch-level* and *prompt-level* information within this class. An overview of our method is shown in Figure 1.

3.3. RLVR with Shrinkage Baseline

Under the standard value-function-baseline framework introduced in Section 3.1, the analysis of the prior section motivates using a shrinkage baseline as a strict baseline-MSE improvement over both per-prompt and batch-mean baselines, and consequently as a route to lower-variance policy gradients. However, a critical additional requirement in policy gradient methods is *unbiasedness*. The naive shrinkage baseline in Equation (9) cannot be used directly, since it is correlated with the rewards r_i^j that appear in the gradient estimator, and thus would introduce bias.

In this section, we consider the general case that prompts are sampled from $\mathcal{D}$ instead of being fixed, and derive a more general shrinkage baseline with unbiasedness guarantee. To guarantee independence between the baseline and each individual reward, we adopt a two-level leave-one-out construction in the spirit of RLOO. For each prompt x_i and response y_i^j , define

$$\hat{\mu}_i^{-j} := \frac{1}{m-1} \sum_{j' \neq j} r_i^{j'}, \quad \hat{\mu}_{-i} := \frac{1}{n-1} \sum_{k \neq i} \hat{\mu}_k, \quad (10)$$

$$b_i^{j, \text{JS2}} := (1 - \lambda_i^j) \hat{\mu}_i^{-j} + \lambda_i^j \hat{\mu}_{-i}. \quad (11)$$

Compared to Equation (9), Equation (11) replaces both the prompt-level and batch-level means with leave-one-out counterparts, ensuring that $b_i^{j, \text{JS2}}$ is independent of the held-out reward r_i^j . Furthermore, we allow the shrinkage coefficient to vary by sample, i.e., each b_i^j uses its own λ_i^j , which can also be chosen independently of r_i^j . With these modifications, the resulting estimator yields an unbiased policy gradient.

The optimal shrinkage coefficient has essentially the same structure as in the naive shrinkage baseline estimator. Define $\mu(x) := \mathbb{E}_{y \sim \pi(\cdot|x)}[r(x, y)]$ and $\sigma^2(x) := \text{Var}_{y \sim \pi(\cdot|x)}[r(x, y)]$. The following theorem provides its precise expression.

Theorem 1. Let $v_2 = \frac{1}{m-1} \mathbb{E}_{x \sim \mathcal{D}}[\sigma^2(x)]$ and $s_2 = \text{Var}_{x \sim \mathcal{D}}[\mu(x)]$. Then, within the class of leave-one-out linear shrinkage baselines of the form Equation (11), the per-sample shrinkage coefficient that minimizes the batch baseline-MSE in Equation (6) is the same across all prompts i and samples j , and is given by

$$(\lambda_i^j)^* = \frac{n-1}{n} \cdot \frac{v_2}{s_2 + v_2}. \quad (12)$$

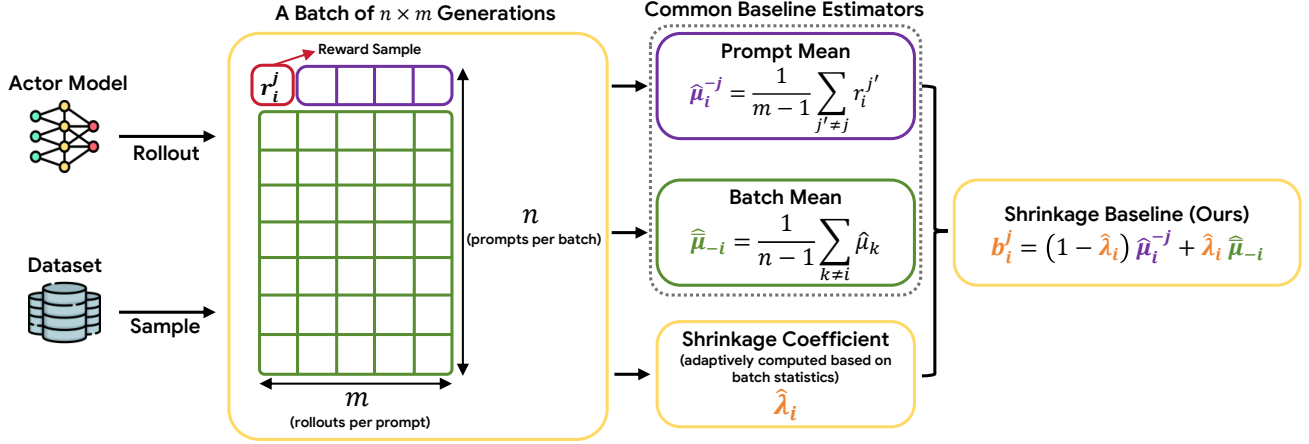


Figure 1. Overview of using the Shrinkage Baseline in RLVR of large reasoning models. Consider a step in RLVR with n question prompts, each generating m responses. For every response, our method computes a leave-one-out prompt-level reward mean $\hat{\mu}_i^{-j}$ and a leave-one-out batch-level reward mean $\hat{\mu}_{-i}$. It then estimates an optimal shrinkage coefficient $\hat{\lambda}_i$ from reward-sample statistics. These components are combined to produce a variance-reduced baseline b_i^j . By lowering the variance in policy-gradient estimation, the shrinkage baseline enables more stable and effective reinforcement learning for large reasoning models.

Here $v_2 = \frac{1}{m-1} \mathbb{E}_{x \sim \mathcal{D}}[\sigma^2(x)]$ measures the *expected variance* of the estimator $\hat{\mu}_i^{-j}$, while $s_2 = \text{Var}_{x \sim \mathcal{D}}[\mu(x)]$ quantifies the variability of the true value functions across prompts. In Appendix A.3, we provide a proof of Theorem 1. Notably, Theorem 1 does not rely on any parametric assumptions on the joint distribution of prompts and rewards.

- When $v_2 \gg s_2$, the **per-prompt estimates are highly noisy**. This typically occurs when only few rollouts per prompt are available, and in such cases, **our shrinkage baseline naturally degrades to the global batch mean baseline**.
- When $s_2 \gg v_2$, the **value functions vary substantially across prompts**. In such cases, the optimal λ is close to zero, and **our shrinkage baseline naturally degrades to the prompt-level mean baseline**.

Thus, $(\lambda_i^j)^*$ achieves the optimal trade-off between variance reduction and bias control, while preserving the independence condition required for unbiased policy gradients.

Implementation details In practice, we first estimate $\hat{v}_{-i}$ and $\hat{s}_{-i}$ from statistics among batch leave-one-out reward samples:

$$\hat{v}_{-i} = \frac{1}{n-1} \sum_{k \neq i} \left(\frac{1}{m(m-1)} \sum_{j=1}^m (r_k^j - \hat{\mu}_k)^2 \right), \quad (13)$$

$$\hat{s}_{-i} = \frac{1}{n-1} \sum_{k \neq i} (\hat{\mu}_k - \hat{\mu}_{-i})^2. \quad (14)$$

Given these plug-in estimates, the per-prompt shrinkage coefficient becomes

$$\hat{\lambda}_i = \frac{n-1}{n} \cdot \frac{\hat{v}_{-i}}{\hat{v}_{-i} + \hat{s}_{-i}}. \quad (15)$$

Finally, combining the local and global averages (Equation (10)) with the estimated $\hat{\lambda}_i$ in Equation (15) yields the *shrinkage baseline*:

$$b_i^j = (1 - \hat{\lambda}_i) \hat{\mu}_i^{-j} + \hat{\lambda}_i \hat{\mu}_{-i}. \quad (16)$$

It is worth noting that b_i^j here does not depend on the response y_i^j , so it satisfies the condition of Proposition 1, and thus the resulting gradient is **unbiased**.

4. Experimental Analysis

In this section, we empirically evaluate the effectiveness of the proposed shrinkage baseline (also named the James-Stein baseline) in reinforcement finetuning of large reasoning models. Through experimental analysis, we show that the shrinkage baseline benefits reinforcement learning by reducing the variance of the policy gradient. For the main experimental setup, we adopt the GRPO algorithm (Shao et al., 2024) without advantage normalization as recommended by a recent large-scale empirical investigation (Khatri et al., 2026), and with leave-one-out reward centering, also known as RLOO (Ahmadian et al., 2024).

4.1. Mathematical Reasoning

We first evaluate the shrinkage baseline on mathematical reasoning tasks. In this section, we use Qwen2.5-Math-1.5B (Team, 2024), Qwen2.5-Math-7B (Team, 2024), and Qwen3-4B-Base (Yang et al., 2025) as base models and train on the DAPO17k (Yu et al., 2025) and Math12k (Hendrycks et al., 2021) datasets. During training, we use 64 questions per batch with 4 rollouts per question. For the Qwen2.5 math models, we set the maximum token length

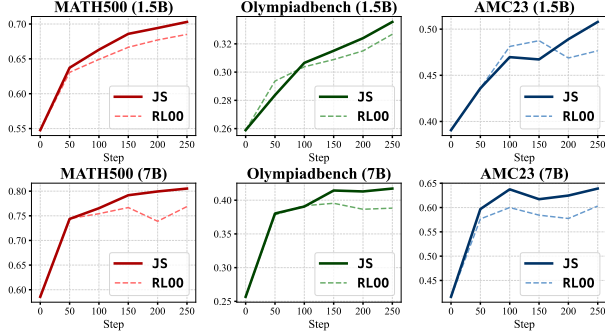


Figure 2. Test accuracy (Pass@1) of the shrinkage baseline and the RLOO baseline (Ahmadian et al., 2024) on Qwen2.5 math models trained on the DAPO17k dataset. The shrinkage baseline significantly outperforms RLOO across different models and benchmarks.

to 2048 and evaluate on three commonly used benchmarks: MATH500 (Hendrycks et al., 2021), OlympiadBench (He et al., 2024), and AMC23 (math ai, 2025). For the Qwen3-4B-Base model, we extend the maximum token length to 3072 and the total training steps to 1500, and evaluate on AIME24 (Hugging Face H4, 2024), AIME25 (OpenCompass Team, 2025) and AMC23 (math ai, 2025) datasets.

Figure 2 shows the test-set accuracy of math reasoning on Qwen2.5 math models, measured by the average Pass@1 over 16 samples, and Figure 3 presents the training reward curve and test-set accuracy of the Qwen3-4B-Base model. Compared to the RLOO baseline (Ahmadian et al., 2024), which uses only the leave-one-out mean reward within each question without shrinkage toward the batch mean, our shrinkage baseline achieves substantial improvements across different models and benchmarks, yielding 1.1%–4.3% gains in Pass@1 along with faster reward growth during training. Additional experimental details and training results are provided in Appendix D.5.1.

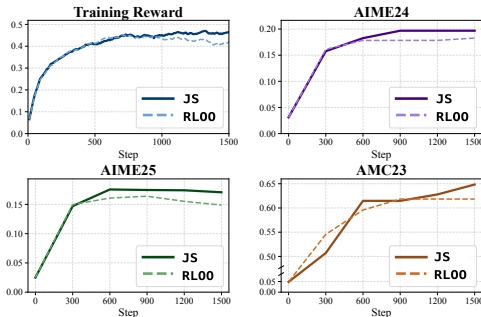


Figure 3. Training reward and test accuracy of the shrinkage baseline and RLOO on the Qwen3-4B-Base model trained on the DAPO17k dataset.

4.2. Logic Puzzle Reasoning

We then apply the shrinkage baseline to various logic puzzle reasoning tasks. Following prior work (Pan et al., 2025; Chen et al., 2025b; Stojanovski et al., 2025), we adopt three settings that are suitable for reinforcement finetuning in terms of model capability and task difficulty: Qwen2.5-7B-Instruct (Team, 2024) on Knights-and-Knives (KnK) (Stojanovski et al., 2025), Qwen2.5-3B (Team, 2024) on Countdown (Gandhi et al., 2024; Pan et al., 2025), and Ministral-8B-Instruct (Jiang et al., 2023) on Maze (Chen et al., 2025b). For each model, we train for at least 200 steps and evaluate on a held-out test set of at least 200 problems drawn from the same distribution as the training data. Additionally, we train Qwen2.5-1.5B-Instruct (Team, 2024) on the KnK dataset for up to 1000 steps to further validate the effectiveness of our method. Detailed descriptions of each puzzle, along with additional experimental results, are provided in Appendix D.5.2.

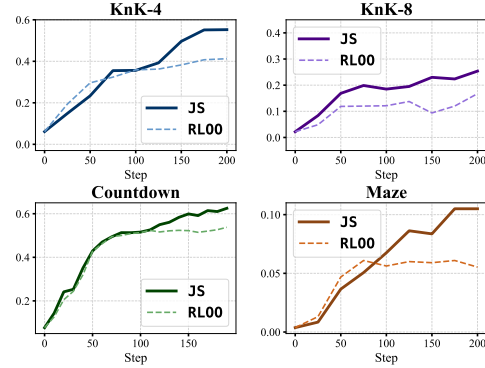


Figure 4. Average test scores of the shrinkage baseline and RLOO on logic puzzle reasoning tasks. The shrinkage baseline outperforms RLOO across all tasks and models. The number following Knights-and-Knives (KnK) indicates the number of people in the puzzle; larger numbers correspond to higher difficulty.

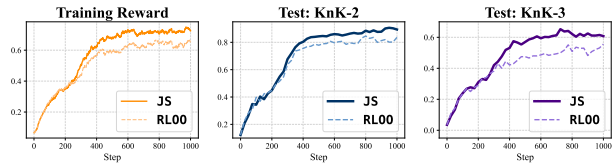


Figure 5. Training reward and average test scores of the shrinkage baseline and RLOO on the Qwen2.5-1.5B-Instruct model trained on the KnK dataset.

Figure 4 plots the average test scores over training steps across different logic puzzles, and Figure 5 shows the training rewards and test scores of the long KnK run. Compared to the RLOO baseline, the shrinkage baseline yields substantial improvements, achieving 4.8%–15.2% gains in Pass@1 on the test sets. These results demonstrate that the shrinkage baseline enables more stable and effective parameter updates during reinforcement learning of reasoning LLMs.

4.3. Multimodal Reasoning

We also leverage the shrinkage baseline for visual reasoning tasks. Following the training recipe in verl (Volcengine, 2024), we train Qwen2.5-VL-3B-Instruct (Bai et al., 2025) and Qwen2.5-VL-7B-Instruct (Bai et al., 2025) on the Geometry3k (Lu et al., 2021) dataset for 600 gradient steps. This dataset contains 2,101 training examples of challenging geometry questions that require joint understanding and reasoning over both text and image information. During training, we evaluate on the test split of Geometry3k which consists of 601 questions for every 20 gradient steps. As shown in Figure 6, the shrinkage baseline consistently outperforms the RLOO baseline throughout training. Additional details are provided in Appendix D.5.3.

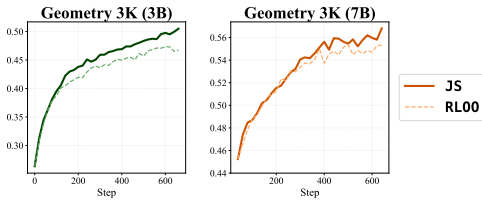


Figure 6. Test scores of the shrinkage baseline and RLOO on Qwen2.5-VL-3B-Instruct and Qwen2.5-VL-7B-Instruct trained on the Geometry3k dataset.

4.4. Reinforcement Learning with Human Feedback

Beyond reasoning tasks, we further extend our shrinkage baseline to another important LLM post-training setting: reinforcement learning from human feedback (RLHF). The goal of RLHF is to align an LLM’s outputs with human preferences, which are usually modeled by a Bradley–Terry reward model, instead of the binary rewards used in the previous three experiments. Following previous work (Hu et al., 2025; Tiapkin et al., 2025; Zhang et al., 2025b; Leng et al., 2025), we use a comprehensive training dataset (Dong et al., 2024) that contains 179k prompts from six widely used RLHF datasets: UltraFeedback (Cui et al., 2024), HelpSteer (Wang et al., 2024), OpenOrca (Lian et al., 2023), UltraInteract (Yuan et al., 2025a), DIBT-10K (Data is Better Together community, 2024), and Capybara Preferences (Argilla et al., 2024). We train a Llama-3.2-3B-Instruct model (Grattafiori et al., 2024) for one epoch using an 8B reward model (OpenRLHF, 2025) trained on 700k preference pairs, and evaluate on three widely used benchmarks: Arena-Hard-v0.1 (Li et al., 2025), Arena-Hard-v2.0 (Li et al., 2025), and Arena-Creative-Writing (Li et al., 2025). As shown in Table 1, the shrinkage baseline consistently outperforms the RLOO baseline on all three benchmarks. These results show that the effectiveness of the shrinkage baseline is not confined to rule-based, binary-reward tasks.

Table 1. RLHF benchmark results (win rates). AH = Arena-Hard, CW = Arena-Creative-Writing. Green values indicate relative improvement over RLOO.

Method	AH-v0.1	AH-v2.0	CW
Llama-3.2-3B-It	26.2%	2.4%	5.2%
RLOO	55.3%	5.6%	20.2%
JS	56.7% +2.5%	7.4% +32.1%	23.4% +15.8%

4.5. Comparison with Other Baselines

We explore the performance of the shrinkage baseline under different numbers of rollouts and systematically compare it with other variance reduction baselines. For computational efficiency, we train the Qwen2.5-0.5B-Instruct (Team, 2024) model on the GSM8k (Cobbe et al., 2021) dataset for 500 steps. For each RL step, we sample 64 questions and vary the number of rollouts (i.e., the number of generations per question) over $\{2, 4, 8\}$. We experiment on different critic-free RLVR baselines, including the GRPO baseline (Shao et al., 2024), RLOO baseline (Ahmadian et al., 2024), ReMax baseline (Li et al., 2024), REINFORCE++ (Hu et al., 2025), batch-level leave-one-out (BLOO), and JS baseline. BLOO computes the baseline as the average of rewards within the batch with the current prompt left out, i.e., it uses $\hat{\mu}_{-i}$ from Equation (16). For each setting, we iterate over 5 random seeds and report the average final accuracy on the GSM8k test split.

Table 2. Final test accuracy (%) across varying numbers of rollouts. Best results are in bold, second-best marked with *, third-best with †.

Baseline	2 Gen	4 Gen	8 Gen
ReMax	54.70 [†]	56.47*	57.76
REINFORCE++	54.82*	55.27	57.30
GRPO	53.68	56.24	58.28 [†]
BLOO	54.19	56.06	57.22
RLOO	54.49	56.34 [†]	58.31*
JS	55.22	57.33	58.93

As shown in Table 2, using the shrinkage baseline consistently achieves the highest evaluation scores across all rollout settings, whereas competing baselines only excel under specific conditions. RLOO and GRPO perform best with 8 rollouts, where prompt-level reward averaging becomes accurate, while REINFORCE++ and BLOO fare relatively better with only 2 rollouts, since GRPO and RLOO suffer from higher variance and batch-level averaging provides more stability. ReMax delivers modest performance across all settings, consistent with the limitations of its greedy decoding design.

4.6. Analysis of Training Dynamics

We provide insights into two key training dynamics that reveal the advantage of using the JS baseline: the *Adaptive Shrinkage* of the James–Stein coefficient and the *Reduced Variance* of the policy gradient. Additional results illustrating these two effects are provided in Section D.4.

Adaptive Shrinkage. The shrinkage coefficient $\hat{\lambda}_i$ in Equation (16) is central to the JS baseline. Figure 7 reports its average value during training across different rollout counts, revealing two key trends: (i) with more rollouts, $\hat{\lambda}_i$ decreases, since intra-prompt estimates become more reliable and the shrinkage baseline naturally approaches the RLOO baseline; and (ii) $\hat{\lambda}_i$ decays over training, as RLVR drives the policy toward greater determinism, reducing the usefulness of cross-prompt references. Together, these behaviors constitute an adaptive shrinkage mechanism that adjusts with both the rollout number and training progress, explaining why the shrinkage baseline consistently outperforms RLOO and BLOO across all rollout settings in Section 4.5.

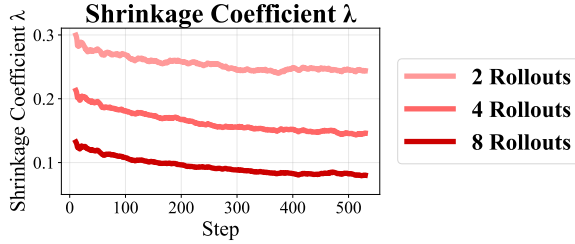


Figure 7. Moving average of the shrinkage coefficient $\hat{\lambda}_i$ during training, for different numbers of rollouts.

Reduced Variance. The variance of the policy gradient in Equation (2) is the key metric reflecting the stability of reinforcement finetuning. To track the gradient variance, we need to build an unbiased estimator of $\text{Var}(g)$ using the observable gradients during training. Following McCanlish et al. (2018), we collect m micro-batches of gradients g_i ($i = 1, \dots, m$) in one training step, then an unbiased estimator for $\text{Var}(g)$ becomes

$$\widehat{\text{Var}}(g) = \frac{1}{m} \cdot \frac{1}{m-1} \sum_{i=1}^m \|g_i - \bar{g}\|^2 \quad (17)$$

$$= \frac{1}{m} \cdot \frac{1}{m-1} \left(\sum_{i=1}^m \|g_i\|^2 - \frac{1}{m} \left\| \sum_{i=1}^m g_i \right\|^2 \right) \quad (18)$$

where $\bar{g} = \frac{1}{m} \sum_{i=1}^m g_i$. We incorporate this estimator during training. Further implementation details are provided in Section D.2.

Figure 8 reports the running average of gradient variance across different experiments and models. Compared to RLOO, training with the shrinkage baseline reduces gradient variance by 11.2%, 17.4%, 31.6%, and 67.1%, respectively, suggesting a more stable training process.

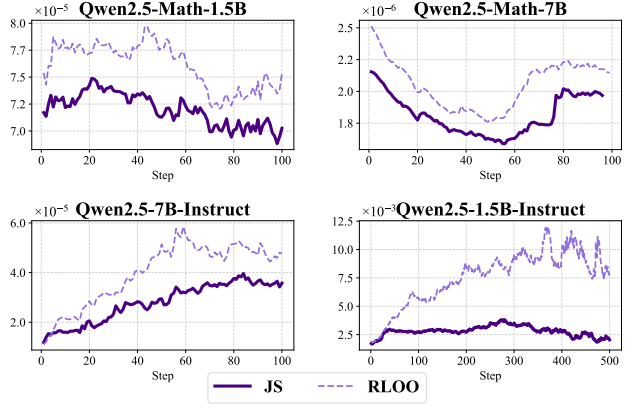


Figure 8. Estimated variance of the policy gradient during training across different models. With the shrinkage baseline, the gradient variance is significantly reduced.

To further measure the variance reduction mechanism, we estimate the ground-truth value function and policy gradient via extensive Monte Carlo sampling on saved checkpoints. As detailed in Appendix B, the shrinkage baseline yields lower mean squared error with respect to the ground-truth value function (Appendix B.1) and produces gradient updates closer to the ground-truth policy gradient in terms of L2 norm (Appendix B.2). In addition, we examine the robustness of the shrinkage baseline under varying training data distributions in Appendix C. Results show that the shrinkage coefficient adapts to dataset difficulty heterogeneity while consistently outperforming the non-shrinkage baseline (Appendix C.1). Moreover, increased semantic heterogeneity primarily affects the magnitude of variance reduction, yet the shrinkage baseline remains optimal (Appendix C.2). Together, these results suggest that the shrinkage baseline provides robust variance reduction across diverse situations in LLM reinforcement learning.

5. Related Work

5.1. Reinforcement Learning with Verifiable Rewards (RLVR)

RLVR refers to a training paradigm where the reward is computed by a rule-based verification function that typically indicates whether the model’s final answer is correct. This approach has proven effective in enhancing the reasoning capabilities of LLMs. The recent success of RLVR is closely tied to advances in reinforcement learning algorithms, which can be broadly categorized into two groups. **Actor-critic methods**, such as PPO (Schulman et al., 2017) and its variants (e.g., VC-PPO (Yuan et al., 2025c), VinePPO (Kazemnejad et al., 2025), VAPO (Yuan et al., 2025b)), rely on training an additional value model (critic) to estimate baselines. While theoretically grounded, these methods incur high computational overhead. **Critic-free methods**,

including RLOO (Ahmadian et al., 2024), ReMax (Li et al., 2024), GRPO (Shao et al., 2024), DAPO (Yu et al., 2025), Dr.GRPO (Liu et al., 2025), GSPO (Zheng et al., 2025) and CISPO (MiniMax, 2025) eliminate the need for a learned value function by directly estimating baselines or advantages from multiple responses to the same prompt. These methods significantly reduce training cost and have become the dominant approach in practical RLVR pipelines. Their effectiveness largely hinges on the quality of the estimated baseline, which serves as a variance reduction tool in policy gradient updates.

While RLVR inherits techniques from classic RL, a distinctive characteristic when applied to LLMs is the use of large batch sizes but small rollout numbers per prompt. For example, rollout numbers of 2, 4, or 8 are typically adopted (Shao et al., 2024; volcengine, 2025; OpenRLHF, 2025; Zhang et al., 2025a; Chen et al., 2025a; Kang et al., 2025; Phan et al., 2025), and even large-scale industrial post-training practices (Yang et al., 2025; Guo et al., 2025; DeepSeek-AI et al., 2025) rarely sample more than 32 responses per prompt due to the high cost of LLM inference.

5.2. Baselines in Policy Gradient Methods

The use of baselines in policy gradient methods was originally introduced (Williams, 1992) as a variance reduction technique without introducing bias. Early work formalized this as a control variate problem, showing that the optimal constant baseline is the average return (Weaver & Tao, 2001), while state-dependent baselines can further reduce variance (Greensmith et al., 2004). Actor-critic methods (Barto et al., 1989) extend this idea by learning value function approximations, and techniques such as generalized advantage estimation (GAE) (Schulman et al., 2016) trade off bias and variance to improve stability and sample efficiency. More recent work explores state-action-dependent baselines: Q-Prop (Gu et al., 2017) leverages off-policy critics as control variates, and action-dependent factorized baselines (Wu et al., 2018) exploit policy structure to achieve lower variance in high-dimensional settings. Other methods use Stein’s identity to learn expressive baselines with action dependency (Liu et al., 2018). However, a later empirical study (Tucker et al., 2018) suggests that when value functions are well-approximated, simple state-dependent baselines can match or outperform more complex alternatives. In summary, an effective baseline should minimize variance, maintain zero or low bias, and be robust to implementation, with growing consensus emphasizing careful design over complexity.

5.3. Stein’s Paradox

Stein’s paradox (Stein et al., 1956) highlights a striking fact in multivariate decision theory: when estimating the mean

vector $\theta \in \mathbb{R}^p$ from a single observation $X \sim \mathcal{N}_p(\theta, \sigma^2 I)$ under squared-error loss, the usual estimator $\hat{\theta} = X$ (equivalently, the coordinatewise MLE) is inadmissible as soon as $p \geq 3$. In contrast, the James–Stein (JS) estimator (Stein et al., 1956; James et al., 1961) achieves uniformly smaller risk by shrinking the observation toward a common target, thereby “borrowing strength” across coordinates and reducing overall mean squared error.

$$\hat{\theta}_{\text{JS}}(X) = \mu + \left(1 - \frac{(p-2)\sigma^2}{\|X - \mu\|_2^2}\right)(X - \mu) \quad (19)$$

This phenomenon sparked applications of shrinkage rules and their decision-theoretic properties, including empirical Bayes interpretations and data-driven shrinkage (Efron & Morris, 1973), minimax shrinkage families (Baranchik, 1970), and admissibility characterizations via connections to potential theory and diffusions (Brown, 1971; Berger, 1976). Classical James–Stein results are developed within multivariate normal frameworks. We refer to our proposed shrinkage baseline as the JS baseline to highlight this conceptual similarity, though the rule we derive is tailored to RLVR and differs from the classical formula.

6. Conclusion

This paper identifies an underexploited opportunity to reduce policy-gradient variance in critic-free RLVR and introduces a statistically principled, hyperparameter-free estimator that fully leverages the batch structure. The resulting shrinkage policy gradient estimator provides provable variance reduction while remaining a simple, drop-in replacement for existing RL pipelines. Theoretical derivations prove that the estimator has lower expected mean squared error than non-shrinkage counterparts, provably leading to lower policy gradient variance. Empirical results validate that the estimator consistently enhances RLVR training performance across different models, tasks, and numbers of rollouts. Ablation analysis shows that the shrinkage baseline has broad applicability in reducing the variance of the policy gradient estimator, regardless of task difficulty or semantic heterogeneity. We expect the insights behind this approach to inspire further improvements in critic-free reinforcement learning.

Impact Statement

This paper reduces policy-gradient variance in reinforcement learning of LLMs without additional hyperparameters or computation, which has the potential to lower the engineering and compute cost of post-training large reasoning models. Other than that, this paper presents work whose goal is to advance the field of Machine Learning. There are many potential overall societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgments

This work was partially supported by the National Science Foundation under Grants CCF-2106778 and DMS-2134080. This work used DeltaAI at NCSA through allocation CIS250428: Online Curriculum Learning for Large Language Models Reasoning and CIS250567: Training Reasoning Models for Best-of-N Performance from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program (Boerner et al., 2023), which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

References

- Ahmadian, A., Cremer, C., Gallé, M., Fadaee, M., Kreutzer, J., Pietquin, O., Üstün, A., and Hooker, S. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12248–12267, 2024.
- Argilla, KAIST AI, and Hugging Face. Capybara-preferences dataset. <https://huggingface.co/datasets/argilla/Capybara-Preferences>, 2024. Synthetic preference dataset built with distilabel on top of LDJnr/Capybara. Accessed: 2026-05-29.
- Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., Zhong, H., Zhu, Y., Yang, M., Li, Z., Wan, J., Wang, P., Ding, W., Fu, Z., Xu, Y., Ye, J., Zhang, X., Xie, T., Cheng, Z., Zhang, H., Yang, Z., Xu, H., and Lin, J. Qwen2.5-VL technical report, 2025. URL <https://arxiv.org/abs/2502.13923>.
- Baranchik, A. J. A family of minimax estimators of the mean of a multivariate normal distribution. *The Annals of Mathematical Statistics*, 41(2):642–645, April 1970. doi: 10.1214/aoms/1177697104.
- Barto, A. G., Sutton, R. S., and Watkins, C. *Learning and sequential decision making*, volume 89. University of Massachusetts Amherst, MA, 1989.
- Berger, J. O. Admissible minimax estimation of a multivariate normal mean with arbitrary quadratic loss. *The Annals of Statistics*, 4(1):223–226, January 1976. doi: 10.1214/aos/1176343356.
- Brown, L. D. Admissible estimators, recurrent diffusions, and insoluble boundary value problems. *The Annals of Mathematical Statistics*, 42(3):855–903, 1971.
- Chen, X., Zhang, R., Jiang, D., Zhou, A., Yan, S., Lin, W., and Li, H. MINT-CoT: Enabling interleaved visual tokens in mathematical chain-of-thought reasoning. In *Advances in Neural Information Processing Systems*, 2025a.
- Chen, Z., Qin, X., Wu, Y., Ling, Y., Ye, Q., Zhao, W. X., and Shi, G. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models. *arXiv preprint arXiv:2508.10751*, 2025b.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Cui, G., Yuan, L., Ding, N., Yao, G., He, B., Zhu, W., Ni, Y., Xie, G., Xie, R., Lin, Y., Liu, Z., and Sun, M. UL-TRAFEEEDBACK: Boosting language models with scaled AI feedback. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 9722–9744. PMLR, 2024. URL <https://proceedings.mlr.press/v235/cui24f.html>.
- Data is Better Together community. DIBT/10k_prompts_ranked: Community-ranked prompt dataset. https://huggingface.co/datasets/data-is-better-together/10k_prompts_ranked, 2024. Prompt dataset co-created by Argilla, Hugging Face, and the Prompt Collective community. Accessed: 2026-05-29.
- DeepSeek-AI, Liu, A., Mei, A., Lin, B., Xue, B., Wang, B., Xu, B., Wu, B., Zhang, B., Lin, C., Dong, C., Lu, C., Zhao, C., Deng, C., Xu, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Li, E., Zhou, F., Lin, F., Dai, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Li, H., Liang, H., Wei, H., Zhang, H., Luo, H., Ji, H., Ding, H., Tang, H., Cao, H., Gao, H., Qu, H., Zeng, H., Huang, J., Li, J., Xu, J., Hu, J., Chen, J., Xiang, J., Yuan, J., Cheng, J., Zhu, J., Ran, J., Jiang, J., Qiu, J., Li, J., Song, J., Dong, K., Gao, K., Guan, K., Huang, K., Zhou, K., Huang, K., Yu, K., Wang, L., Zhang, L., Wang, L., Zhao, L., Yin, L., Guo, L., Luo, L., Ma, L., Wang, L., Zhang, L., Di, M. S., Xu, M. Y., Zhang, M., Zhang, M., Tang, M., Zhou, M., Huang, P., Cong, P., Wang, P., Wang, Q., Zhu, Q., Li, Q., Chen, Q., Du, Q., Xu, R., Ge, R., Zhang, R., Pan, R., Wang, R., Yin, R., Xu, R., Shen, R., Zhang, R., Liu, S. H., Lu, S., Zhou, S., Chen, S., Cai, S., Chen, S., Hu, S., Liu, S., Hu, S., Ma, S., Wang, S., Yu, S., Zhou, S., Pan, S., Zhou, S., Ni, T., Yun, T., Pei, T., Ye, T., Yue, T., Zeng, W., Liu, W., Liang, W., Pang, W., Luo, W., Gao, W., Zhang, W., Gao, X., Wang, X., Bi, X., Liu, X., Wang, X., Chen, X., Zhang, X., Nie, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Li, X., Yang, X., Li, X., Chen, X., Su, X., Pan, X., Lin, X., Fu, X., Wang, Y. Q., Zhang, Y., Xu, Y., Ma, Y., Li, Y., Li, Y., Zhao, Y., Sun, Y., Wang, Y., Qian, Y., Yu, Y., Zhang, Y., Ding, Y., Shi, Y., Xiong, Y., He, Y., Zhou, Y., Zhong,

- Y., Piao, Y., Wang, Y., Chen, Y., Tan, Y., Wei, Y., Ma, Y., Liu, Y., Yang, Y., Guo, Y., Wu, Y., Wu, Y., Cheng, Y., Ou, Y., Xu, Y., Wang, Y., Gong, Y., Wu, Y., Zou, Y., Li, Y., Xiong, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Zhao, Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Huang, Z., Wu, Z., Li, Z., Zhang, Z., Xu, Z., Wang, Z., Gu, Z., Zhu, Z., Li, Z., Zhang, Z., Xie, Z., Gao, Z., Pan, Z., Yao, Z., Feng, B., Li, H., Cai, J. L., Ni, J., Xu, L., Li, M., Tian, N., Chen, R. J., Jin, R. L., Li, S. S., Zhou, S., Sun, T., Li, X. Q., Jin, X., Shen, X., Chen, X., Song, X., Zhou, X., Zhu, Y. X., Huang, Y., Li, Y., Zheng, Y., Zhu, Y., Ma, Y., Huang, Z., Xu, Z., Zhang, Z., Ji, D., Liang, J., Guo, J., Chen, J., Xia, L., Wang, M., Li, M., Zhang, P., Chen, R., Sun, S., Wu, S., Ye, S., Wang, T., Xiao, W. L., An, W., Wang, X., Sun, X., Wang, X., Tang, Y., Zha, Y., Zhang, Z., Ju, Z., Zhang, Z., and Qu, Z. DeepSeek-V3.2: Pushing the frontier of open large language models, 2025. URL <https://arxiv.org/abs/2512.02556>.
- Dong, H., Xiong, W., Pang, B., Wang, H., Zhao, H., Zhou, Y., Jiang, N., Sahoo, D., Xiong, C., and Zhang, T. RLHF workflow: From reward modeling to online RLHF. *Transactions on Machine Learning Research*, 2024. URL <https://arxiv.org/abs/2405.07863>. Published in TMLR, 2024.
- Efron, B. and Morris, C. Stein’s estimation rule and its competitors—an empirical bayes approach. *Journal of the American Statistical Association*, 68(341):117–130, 1973.
- Gandhi, K., Lee, D., Grand, G., Liu, M., Cheng, W., Sharma, A., and Goodman, N. D. Stream of search (SoS): Learning to search in language. In *Conference on Language Modeling (COLM)*, 2024.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Greensmith, E., Bartlett, P. L., and Baxter, J. Variance reduction techniques for gradient estimates in reinforcement learning. *Journal of Machine Learning Research*, 5(Nov): 1471–1530, 2004.
- Gu, S., Lillicrap, T., Ghahramani, Z., Turner, R. E., and Levine, S. Q-Prop: Sample-efficient policy gradient with an off-policy critic. In *International Conference on Learning Representations*, 2017. Oral.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pp. 1861–1870. PMLR, 2018.
- He, C., Luo, R., Bai, Y., Hu, S., Thai, Z., Shen, J., Hu, J., Han, X., Huang, Y., Zhang, Y., et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3828–3850, 2024.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- Hu, J., Liu, J. K., Xu, H., and Shen, W. REINFORCE++: Stabilizing critic-free policy optimization with global advantage normalization. *arXiv preprint arXiv:2501.03262*, 2025.
- Hugging Face H4. Aime 2024 dataset. https://huggingface.co/datasets/HuggingFaceH4/aime_2024, 2024. 30 problems from AIME I & II (2024). Accessed: 2025-10-21.
- James, W., Stein, C., et al. Estimation with quadratic loss. In *Proceedings of the fourth Berkeley symposium on mathematical statistics and probability*, volume 1, pp. 361–379. University of California Press, 1961.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de Las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mistral 7b. *ArXiv*, abs/2310.06825, 2023. URL <https://api.semanticscholar.org/CorpusID:263830494>.
- Kang, X., Wu, S., Wang, Z., Liu, Y., Jin, X., Huang, K., Wang, W., Yue, Y., Huang, X., and Wang, Q. Can GRPO boost complex multimodal table understanding? In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 12631–12644, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.637. URL <https://aclanthology.org/2025.emnlp-main.637/>.
- Kazemnejad, A., Aghajohari, M., Portelance, E., Sordoni, A., Reddy, S., Courville, A., and Roux, N. L. VinePPO:

- Refining credit assignment in RL training of LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 29557–29590, 2025.
- Khatri, D., Madaan, L., Tiwari, R., Bansal, R., Duvvuri, S. S., Zaheer, M., Dhillon, I. S., Brandfonbrener, D., and Agarwal, R. The art of scaling reinforcement learning compute for LLMs. In *International Conference on Learning Representations*, 2026. Oral.
- Leng, J., Huang, C., Zhu, B., and Huang, J. Taming overconfidence in llms: Reward calibration in rlhf. In *International Conference on Learning Representations*, volume 2025, pp. 16484–16517, 2025.
- Li, T., Chiang, W.-L., Frick, E., Dunlap, L., Wu, T., Zhu, B., Gonzalez, J. E., and Stoica, I. From crowdsourced data to high-quality benchmarks: Arena-Hard and BenchBuilder pipeline. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 34209–34231, 2025.
- Li, Z., Xu, T., Zhang, Y., Lin, Z., Yu, Y., Sun, R., and Luo, Z.-Q. ReMax: A simple, effective, and efficient reinforcement learning method for aligning large language models. In *Proceedings of the 41st International Conference on Machine Learning*, *Proceedings of Machine Learning Research*, 2024.
- Lian, W., Goodson, B., Pentland, E., Cook, A., Vong, C., and Teknium. Openorca: An open dataset of GPT augmented FLAN reasoning traces. <https://huggingface.co/datasets/Open-Orca/OpenOrca>, 2023. Hugging Face dataset.
- Liu, H., Feng, Y., Mao, Y., Zhou, D., Peng, J., and Liu, Q. Action-dependent control variates for policy optimization via stein’s identity. In *International Conference on Learning Representations*, 2018.
- Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025.
- Lu, P., Gong, R., Jiang, S., Qiu, L., Huang, S., Liang, X., and Zhu, S.-C. Inter-GPS: Interpretable geometry problem solving with formal language and symbolic reasoning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL-IJCNLP)*, 2021.
- math ai. AMC23 Dataset (Hugging Face). <https://huggingface.co/datasets/math-ai/amc23>, 2025. Accessed: 2025-09-18.
- McCandlish, S., Kaplan, J., Amodei, D., and Team, O. D. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018.
- MiniMax. MiniMax-M1: Scaling test-time compute efficiently with lightning attention, 2025. URL <https://arxiv.org/abs/2506.13585>.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., and Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, pp. 1928–1937. PMLR, 2016.
- OpenAI. Learning to reason with llms. <https://openai.com/index/learning-to-reason-with-llms/>, 2024. Accessed: 2025-05-15.
- OpenCompass Team. Aime 2025 dataset. <https://huggingface.co/datasets/opencompass/AIME2025>, 2025. 30 problems from AIME 2025 I & II. Accessed: 2025-10-21.
- OpenRLHF. A Training Recipe for Llama-3.1-8B on GRPO in OpenRLHF. https://github.com/OpenRLHF/OpenRLHF/blob/dda5a145c6e8b2fe74692325b6c12ed60e253745/examples/scripts/train_grpo_ray_hybrid_engine.sh#L28, 2025. Accessed: 2025-05-18.
- OpenRLHF. Llama-3-8b-rm-700k. <https://huggingface.co/OpenRLHF/Llama-3-8b-rm-700k>, 2025. Llama 3 8B reward model trained on 700K preference pairs.
- Pan, J., Zhang, J., Wang, X., Yuan, L., Peng, H., and Suhr, A. TinyZero. <https://github.com/Jiayi-Pan/TinyZero>, 2025. Accessed: 2025-01-24.
- Phan, H., Yang, X., Yao, K., Zhang, J., Bi, S., Tang, X., Khabsa, M., Liu, L., and Lei, D. Beyond reasoning gains: Mitigating general capabilities forgetting in large reasoning models, 2025. URL <https://arxiv.org/abs/2510.21978>.
- Schulman, J., Levine, S., Abbeel, P., Jordan, M., and Moritz, P. Trust region policy optimization. In *International Conference on Machine Learning*, pp. 1889–1897. PMLR, 2015.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. In *Conference on Language Modeling (COLM)*, 2024.
- Stein, C. et al. Inadmissibility of the usual estimator for the mean of a multivariate normal distribution. In *Proceedings of the Third Berkeley symposium on mathematical statistics and probability*, volume 1, pp. 197–206, 1956.
- Stojanovski, Z., Stanley, O., Sharratt, J., Jones, R., Adefioye, A., Kaddour, J., and Köpf, A. Reasoning gym: Reasoning environments for reinforcement learning with verifiable rewards. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2025. Spotlight.
- Sutton, R. S. and Barto, A. G. Actor–critic methods. In *Reinforcement Learning: An Introduction*, Adaptive Computation and Machine Learning, chapter 13.5, pp. 331–332. MIT Press, Cambridge, MA, 2 edition, 2018. ISBN 978-0262039246. URL <https://incompleteideas.net/book/the-book-2nd.html>.
- Sutton, R. S., Barto, A. G., et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- Sutton, R. S., McAllester, D., Singh, S., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In *Advances in Neural Information Processing Systems*, 1999.
- Team, Q. Qwen2.5: A party of foundation models, September 2024. URL <https://qwenlm.github.io/blog/qwen2.5/>.
- Tiapkin, D., Calandriello, D., Belomestny, D., Moulines, E., Naumov, A., Rasul, K., Valko, M., and Menard, P. Proximal point nash learning from human feedback. *arXiv preprint arXiv:2505.19731*, 2025.
- Tucker, G., Bhupatiraju, S., Gu, S., Turner, R., Ghahramani, Z., and Levine, S. The mirage of action-dependent baselines in reinforcement learning. In *International Conference on Machine Learning*, pp. 5015–5024. PMLR, 2018.
- Volcengine. Official GRPO Trainer Script for Qwen2-7B in verl. https://github.com/volcengine/verl/blob/a65c9157bc0b85b64cd753de19f94e80a11bd871/examples/grpo_trainer/run_qwen2-7b_seq_balance.sh, 2024. Accessed: 2025-05-15.
- volcengine. A Training Recipe for DeepSeek-671B on GRPO in verl. https://github.com/volcengine/verl/blob/2617b900eb463efc5b751c4360c4ccce0da775f7/examples/grpo_trainer/run_deepseek671b_math_megatron_80gb.sh#L44, 2025. Accessed: 2025-10-22.
- Wang, Z., Dong, Y., Zeng, J., Adams, V., Sreedhar, M. N., Egert, D., Delalleau, O., Scowcroft, J. P., Kant, N., Swope, A., and Kuchaiev, O. HelpSteer: Multi-attribute helpfulness dataset for SteerLM. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 3371–3384, Mexico City, Mexico, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.185. URL <https://aclanthology.org/2024.naacl-long.185>.
- Weaver, L. and Tao, N. The optimal reward baseline for gradient-based reinforcement learning. In *Proceedings of the Seventeenth Conference on Uncertainty in Artificial Intelligence (UAI)*, 2001.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.
- Wu, C., Rajeswaran, A., Duan, Y., Kumar, V., Bayen, A. M., Kakade, S., Mordatch, I., and Abbeel, P. Variance reduction for policy gradient with action-dependent factorized baselines. In *International Conference on Learning Representations*, 2018.
- Xu, Z. and Ding, Z. Single-stream policy optimization. In *International Conference on Learning Representations*, 2026.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Fan, T., Liu, G., Liu, L., Liu, X., et al. DAPO: An open-source LLM reinforcement learning system at scale. In *Advances in Neural Information Processing Systems*, 2025.
- Yuan, L., Cui, G., Wang, H., Ding, N., Wang, X., Shan, B., Liu, Z., Deng, J., Chen, H., Xie, R., Lin, Y., Liu, Z., Zhou, B., Peng, H., Liu, Z., and Sun, M. Advancing LLM reasoning generalists with preference trees. In *Proceedings of the International Conference on Learning Representations*, 2025a. URL <https://arxiv.org/abs/2404.02078>. Introduces the UltraInteract preference dataset.

Yuan, Y., Yu, Q., Zuo, X., Zhu, R., Xu, W., Chen, J., Wang, C., Fan, T., Du, Z., Wei, X., et al. VAPO: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025b.

Yuan, Y., Yue, Y., Zhu, R., Fan, T., and Yan, L. What’s behind PPO’s collapse in long-CoT? value optimization holds the secret. *arXiv preprint arXiv:2503.01491*, 2025c.

Zhang, J., Huang, J., Yao, H., Liu, S., Zhang, X., Lu, S., and Tao, D. R1-VL: Learning to reason with multimodal large language models via step-wise group relative policy optimization. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025a.

Zhang, Y., Yu, D., Ge, T., Song, L., Zeng, Z., Mi, H., Jiang, N., and Yu, D. Improving LLM general preference alignment via optimistic online mirror descent. In *Advances in Neural Information Processing Systems*, 2025b. Spotlight.

Zheng, C., Liu, S., Li, M., Chen, X.-H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., Zhou, J., and Lin, J. Group sequence policy optimization, 2025. URL <https://arxiv.org/abs/2507.18071>.

A. Theoretical Proof

In this section, we use the notation $\mathbf{Y} - y_i^j$ to denote the set of all $y_{i'}^{j'}$ in which $(i', j') \neq (i, j)$.

A.1. Proof of Proposition 1

Consider gradient update on each sample $\mathbb{E}_{\mathbf{x}, \mathbf{Y}}[(r_i^j - b_i^j) \nabla_{\theta} \log \pi_{\theta}(y_i^j | x_i)]$. We have

$$\begin{aligned}
 & \mathbb{E}_{\mathbf{x}, \mathbf{Y}}[(r_i^j - b_i^j) \nabla_{\theta} \log \pi_{\theta}(y_i^j | x_i)] \\
 &= \mathbb{E}_{\mathbf{x}, \mathbf{Y}}[r_i^j \nabla_{\theta} \log \pi_{\theta}(y_i^j | x_i)] - \mathbb{E}_{\mathbf{x}, \mathbf{Y}}[b_i^j \nabla_{\theta} \log \pi_{\theta}(y_i^j | x_i)] \\
 &= \nabla_{\theta} J(\theta) - \mathbb{E}_{\mathbf{x}, \mathbf{Y} - y_i^j} \mathbb{E}_{y_i^j \sim \pi_{\theta}(\cdot | x_i)} [b_i^j \nabla_{\theta} \log \pi_{\theta}(y_i^j | x_i)] \\
 &= \nabla_{\theta} J(\theta) - \mathbb{E}_{\mathbf{x}, \mathbf{Y} - y_i^j} (b_i^j \mathbb{E}_{y_i^j \sim \pi_{\theta}(\cdot | x_i)} [\nabla_{\theta} \log \pi_{\theta}(y_i^j | x_i)]) \\
 &= \nabla_{\theta} J(\theta) - \mathbb{E}_{\mathbf{x}, \mathbf{Y} - y_i^j} (b_i^j \sum_{y_i^j} [\nabla_{\theta} \pi_{\theta}(y_i^j | x_i)]) \\
 &= \nabla_{\theta} J(\theta).
 \end{aligned}$$

Since this holds for all i, j , we have

$$\begin{aligned}
 \mathbb{E}_{\mathbf{x}, \mathbf{Y}}[g(\mathbf{x}, \mathbf{Y}; \theta)] &= \frac{1}{n} \sum_{i=1}^n \frac{1}{m} \sum_{j=1}^m \mathbb{E}_{\mathbf{x}, \mathbf{Y}}[(r_i^j - b_i^j) \nabla_{\theta} \log \pi_{\theta}(y_i^j | x_i)] \\
 &= \nabla_{\theta} J(\theta).
 \end{aligned}$$

A.2. Proof of Proposition 2

Note that $b_i^{j, \text{JS1}} = b_i^{1, \text{JS1}}$ for all i, j . We can also rewrite the interpolation as

$$b_i^{1, \text{JS1}} = (1 - \frac{n-1}{n} \lambda) \hat{\mu}_i + \frac{n-1}{n} \lambda \hat{\mu}_{-i}.$$

We can let $\gamma = \frac{n-1}{n} \lambda$. So we can rewrite the objective as

$$\begin{aligned}
 \text{MSE}^{\text{relax}} &= \mathbb{E}_{\mathbf{Y}}[\frac{1}{n} \sum_{i=1}^n (\mu_i - b_i^{1, \text{JS1}})^2] \\
 &= \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{\mathbf{Y}}[((1 - \gamma)(\mu_i - \hat{\mu}_i) + \gamma(\mu_i - \hat{\mu}_{-i}))^2] \\
 &= \frac{1}{n} \sum_{i=1}^n ((1 - \gamma)^2 \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_i)^2] + 2\gamma(1 - \gamma) \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_i)(\mu_i - \hat{\mu}_{-i})] + \gamma^2 \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_{-i})^2]) \\
 &= \frac{1}{n} \sum_{i=1}^n ((1 - \gamma)^2 \text{Var}_{\mathbf{Y}}[\hat{\mu}_i] + 2\gamma(1 - \gamma) \mathbb{E}_{\mathbf{Y}}[\mu_i - \hat{\mu}_i] \mathbb{E}_{\mathbf{Y}}[\mu_i - \hat{\mu}_{-i}] + \gamma^2 \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_{-i})^2]) \\
 &= \frac{1}{n} \sum_{i=1}^n ((1 - \gamma)^2 \frac{1}{m} \sigma_i^2 + \gamma^2 \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_{-i})^2]).
 \end{aligned}$$

Let $\bar{\mu}_{-i} := \frac{1}{n-1} \sum_{i' \neq i} \mu_{i'}$. For the second term in the summation, note that

$$\begin{aligned}
 \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_{-i})^2] &= \mathbb{E}_{\mathbf{Y}}[(\mu_i - \bar{\mu}_{-i}) + (\bar{\mu}_{-i} - \hat{\mu}_{-i})]^2 \\
 &= (\mu_i - \bar{\mu}_{-i})^2 + \mathbb{E}_{\mathbf{Y}}[(\bar{\mu}_{-i} - \hat{\mu}_{-i})^2] \\
 &= (\frac{n}{n-1})^2 (\mu_i - \bar{\mu})^2 + \text{Var}_{\mathbf{Y}}[\hat{\mu}_{-i}] \\
 &= (\frac{n}{n-1})^2 (\mu_i - \bar{\mu})^2 + \frac{1}{(n-1)^2} \sum_{i' \neq i} \text{Var}_{\mathbf{Y}}[\hat{\mu}_{i'}].
 \end{aligned}$$

Therefore, we have

$$\begin{aligned}
 & \mathbb{E}[\frac{1}{n} \sum_{i=1}^n (\mu_i - b_i^{1, \text{JS1}})^2] \\
 &= \frac{1}{n} \sum_{i=1}^n ((1 - \gamma)^2 \frac{1}{m} \sigma_i^2 + \gamma^2 [(\frac{n}{n-1})^2 (\mu_i - \bar{\mu})^2 + \frac{1}{(n-1)^2} \sum_{i' \neq i} \frac{1}{m} \sigma_{i'}^2]) \\
 &= \frac{1}{n} (1 - \gamma)^2 \sum_{i=1}^n \frac{1}{m} \sigma_i^2 + \frac{n}{(n-1)^2} \gamma^2 \sum_{i=1}^n (\mu_i - \bar{\mu})^2 + \frac{1}{n(n-1)^2} \gamma^2 \sum_{i=1}^n \sum_{i' \neq i} \frac{1}{m} \sigma_{i'}^2 \\
 &= \frac{1}{n} (1 - \gamma)^2 \sum_{i=1}^n \frac{1}{m} \sigma_i^2 + \frac{n}{(n-1)^2} \gamma^2 \sum_{i=1}^n (\mu_i - \bar{\mu})^2 + \frac{1}{n(n-1)} \gamma^2 \sum_{i=1}^n \frac{1}{m} \sigma_i^2 \\
 &= \frac{n}{n-1} (s + v) \gamma^2 - 2v\gamma + v.
 \end{aligned}$$

This is a quadratic function of γ , and so the global minimum is $\gamma^* := \frac{n-1}{n} \frac{v}{s+v}$.

A.3. Proof of Theorem 1

Recall Equation (11):

$$b_i^{j,JS2} = (1 - \lambda_i^j) \hat{\mu}_i^{-j} + \lambda_i^j \hat{\mu}_{-i}.$$

Since each baseline $b_i^{j,JS2}$ has its own shrinkage coefficient λ_i^j , we only need to minimize each square error term $\mathbb{E}_{\mathbf{x}, \mathbf{Y}}[(\mu_i - b_i^{j,JS2})^2]$ in order to minimize the whole objective MSE.

Then for any i, j , we have

$$\begin{aligned} & \mathbb{E}_{\mathbf{x}, \mathbf{Y}}[(\mu_i - b_i^j)^2] \\ &= \mathbb{E}_{\mathbf{x}, \mathbf{Y}}[(1 - \lambda_i^j)(\mu_i - \hat{\mu}_i^{-j}) + \lambda_i^j(\mu_i - \hat{\mu}_{-i})]^2 \\ &= \mathbb{E}_{\mathbf{x}} \mathbb{E}_{\mathbf{Y}}[(1 - \lambda_i^j)^2(\mu_i - \hat{\mu}_i^{-j})^2 + (\lambda_i^j)^2(\mu_i - \hat{\mu}_{-i})^2 + 2\lambda_i^j(1 - \lambda_i^j)(\mu_i - \hat{\mu}_i^{-j})(\mu_i - \hat{\mu}_{-i})] \\ &= \mathbb{E}_{\mathbf{x}}((1 - \lambda_i^j)^2 \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_i^{-j})^2] + (\lambda_i^j)^2 \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_{-i})^2] + 2\lambda_i^j(1 - \lambda_i^j) \mathbb{E}_{\mathbf{Y}}[\mu_i - \hat{\mu}_i^{-j}] \mathbb{E}_{\mathbf{Y}}[\mu_i - \hat{\mu}_{-i}]) \\ &= \mathbb{E}_{\mathbf{x}}((1 - \lambda_i^j)^2 \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_i^{-j})^2] + (\lambda_i^j)^2 \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_{-i})^2]). \end{aligned}$$

For the first term in the summation, we have

$$\mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_i^{-j})^2] = \text{Var}_{\mathbf{Y}}[\hat{\mu}_i^{-j}] = \frac{1}{m-1} \sigma^2(x_i).$$

Recall that $\sigma^2(x_i) = \text{Var}_{y \sim \pi_\theta(\cdot | x_i)}[r(x_i, y)]$. For the second term, we have

$$\begin{aligned} \mathbb{E}_{\mathbf{Y}}[(\mu_i - \hat{\mu}_{-i})^2] &= \mathbb{E}_{\mathbf{Y}}[(\mu_i - \bar{\mu}_{-i}) + (\bar{\mu}_{-i} - \hat{\mu}_{-i})]^2 \\ &= (\mu_i - \bar{\mu}_{-i})^2 + \text{Var}_{\mathbf{Y}}[\hat{\mu}_{-i}] \\ &= (\mu_i - \bar{\mu}_{-i})^2 + \frac{1}{(n-1)^2} \sum_{i' \neq i} \text{Var}_{\mathbf{Y}}[\hat{\mu}_{i'}] \\ &= (\mu_i - \bar{\mu}_{-i})^2 + \frac{1}{(n-1)^2} \sum_{i' \neq i} \frac{1}{m-1} \sigma^2(x_{i'}) \end{aligned}$$

Combining together, we have

$$\begin{aligned} & \mathbb{E}_{\mathbf{x}, \mathbf{Y}}[(\mu_i - b_i^j)^2] \\ &= (1 - \lambda_i^j)^2 \mathbb{E}_{\mathbf{x}}[\frac{1}{m-1} \sigma^2(x_i)] + (\lambda_i^j)^2 \mathbb{E}_{\mathbf{x}}[(\mu_i - \bar{\mu}_{-i})^2 + \frac{1}{(n-1)^2} \sum_{i' \neq i} \frac{1}{m-1} \sigma^2(x_{i'})] \\ &= \frac{1}{m-1} \mathbb{E}_{x \sim \mathcal{D}}[\sigma^2(x)](1 - \lambda_i^j)^2 + \frac{1}{(n-1)(m-1)} \mathbb{E}_{x \sim \mathcal{D}}[\sigma^2(x)](\lambda_i^j)^2 + \frac{n}{n-1} \text{Var}_{x \sim \mathcal{D}}[\mu(x)](\lambda_i^j)^2 \\ &= \frac{n}{n-1} (s_2 + v_2)(\lambda_i^j)^2 - 2v_2 \lambda_i^j + v_2. \end{aligned}$$

Therefore, optimal λ_i^j is

$$(\lambda_i^j)^* = \frac{n-1}{n} \frac{v_2}{s_2 + v_2}.$$

This holds for all i, j , so we complete the proof.

B. Direct Verification of Variance Reduction

B.1. Reduction of Mean Squared Error with Ground-Truth Value Function

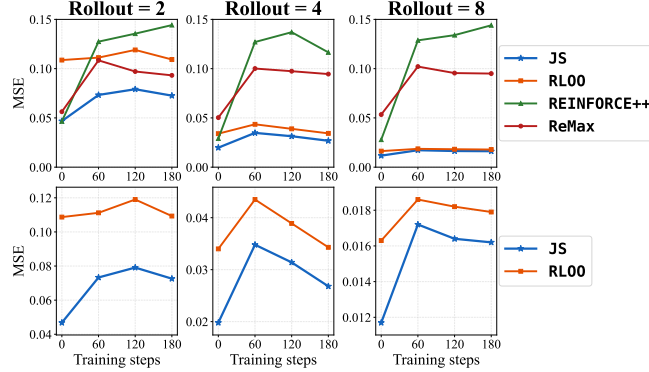


Figure 9. MSE between the value function and estimated baseline under different rollout budgets during training. The results are based on the average of 20 randomly selected batches from DAPO17k, and weights from the experiments on Qwen3-4B-Base. With baseline shrinkage, the estimation is consistently closer to the value function. For 2, 4, and 8 rollouts, the mean squared error of the shrinkage baseline estimator is 39.4%, 25.1%, and 13.4% lower than that of the RLOO estimator, respectively.

Moreover, we estimate the value function of each question by Monte Carlo sampling. For each question, we generate 128 trajectories and compute the average reward as an accurate estimate. After that, we sample another batch of responses under fixed smaller rollout counts, and compute the mean squared error between different baselines and the estimated value over a batch of 64 questions, matching the training setting. The MSE metric for the RLOO, REINFORCE++, ReMax, and JS baselines throughout training is shown in Figure 9. Under different rollout budgets, the JS baseline consistently shows the smallest deviation from the value function compared with RLOO, REINFORCE++, and ReMax. These results support our theoretical derivations and highlight the advantage of using the shrinkage baseline in reinforcement finetuning under varying rollout counts.

B.2. Reduction of Squared L2 Norm with Ground-Truth Policy Gradient

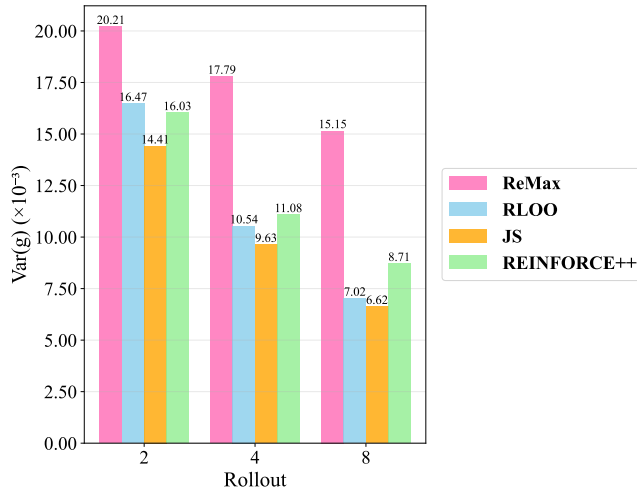


Figure 10. Comparison of different baselines on policy gradient variance, measured by the squared L_2 norm between estimated and ground-truth policy gradients.

We further obtain a precise estimate of the ground-truth policy gradient $\nabla_{\theta} J(\theta)$ and directly compare the variance of the policy gradient under different baselines using Equation (2). We first select 128 questions from the DAPO17k dataset and generate 256 responses for each question using the Qwen3-4B-Base model. For each question, by using all 256 rollouts to compute the policy gradient (Equation (1)), we obtain a precise estimate of the ground-truth $\nabla_{\theta} J(\theta)$. We then randomly

sample 64 questions and n (much smaller than 256) responses for each of them from the large response dataset we created. A small batch of $64 \times n$ samples is formed, similar to RL training setups in practice. Based on this small batch, we compute the policy gradients g_{RLOO} , $g_{\text{REINFORCE++}}$, g_{ReMax} and g_{JS} using different baselines, and compute the variance of the policy gradient by taking the squared L2 norm between g and $\nabla_{\theta} J(\theta)$ (Equation (2)). In Figure 10, we report the average squared L2 norm over 20 small batches with rollout numbers $n = 2, 4, 8$. We can see that the use of the shrinkage baseline reduces the L2 norm between the estimated policy gradient and the ground truth by 12.5%, 8.6%, and 5.7%, respectively, providing direct evidence for improved training stability.

C. Ablation Study

C.1. Ablation Study on Dataset Difficulty Heterogeneity

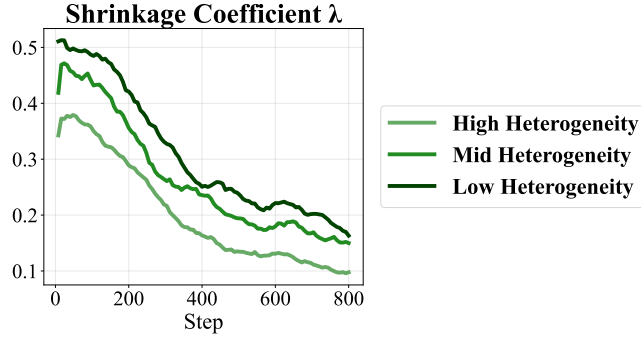


Figure 11. JS shrinkage coefficient during training under different levels of difficulty heterogeneity.

Method	Low	Mid	High
RLOO	20.25	17.69	31.13
JS	25.69	23.85	34.25

Table 3. Average test set accuracy (%) under different heterogeneity settings.

We conduct an ablation study on the effect of the JS baseline under varying levels of dataset heterogeneity. For the Knights-and-Knaves dataset, a larger problem size (i.e., the number of propositions given) indicates higher complexity and difficulty. Therefore, we consider three training settings by mixing data with different difficulty levels: low heterogeneity (KnK-4 & KnK-5), medium heterogeneity (KnK-3 & KnK-7), and high heterogeneity (KnK-2 & KnK-9). For each dataset, we train a Qwen2.5-1.5B-Instruct model (Team, 2024) for 800 gradient steps and evaluate it on 200 questions drawn from the same distribution as the training data. Figure 11 shows the shrinkage coefficient for the different datasets, and Table 3 reports the test accuracy in the different settings. We observe that higher heterogeneity in the difficulty distribution leads to an adaptively smaller shrinkage, i.e., the algorithm behaves closer to vanilla RLOO. This adaptive adjustment mechanism yields an approximately optimal baseline across different training data distributions, resulting in consistent performance gains.

C.2. Ablation Study on Semantic Heterogeneity

Dataset	Pass Rate (%)	λ
KnK-4	13.35	0.43
Countdown-4	14.62	0.41

Table 4. Qwen2.5-7B-Instruct pass rate (%) and shrinkage coefficient (λ) on two datasets.

In addition, we analyze how semantic heterogeneity of prompts affects variance reduction when using shrinkage baselines. We select three training datasets from our earlier experiments: KnK-4, Countdown-4, and a mixed dataset formed by combining 50% KnK-4 and 50% Countdown-4 problems. We run all experiments on Qwen2.5-7B-Instruct. By design, KnK-4 and Countdown-4 have similar pass rates and similar shrinkage coefficients on this model (Table 4). Therefore, mixing the two tasks **increases prompt semantic heterogeneity** while keeping the **reward distribution and shrinkage coefficient approximately matched**. Following the procedure in Section B.2, we estimate the variance of the policy-gradient estimator on Qwen2.5-7B-Instruct under different shrinkage coefficients λ for all three datasets, with a fixed rollout number of 4. As shown in Figure 12, relative to the RLOO baseline ($\lambda = 0.0$), JS shrinkage with $\lambda = 0.4$ reduces policy-gradient variance by 14.7% on KnK-4 and 11.8% on Countdown-4. On the mixed dataset, JS shrinkage still reduces variance, but the improvement is smaller (5.32%). However, $\lambda \approx 0.4$ remains the variance-minimizing choice in all three settings. These results suggest that semantic heterogeneity mainly influences the magnitude of variance reduction, while a James–Stein-style

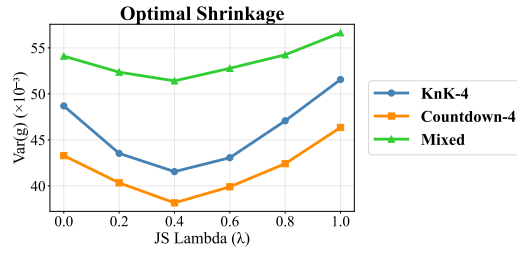


Figure 12. Effect of semantic heterogeneity on policy-gradient variance reduction with shrinkage baselines.

interpolation between the prompt-level mean and the batch-level mean continues to minimize gradient variance, regardless of the semantic heterogeneity of the training data.

D. Experimental Details

D.1. Code Implementation

Below is the core python implementation of computing the advantage according to the shrinkage baseline. It only requires several lines of code and negligible additional computation.

Listing 1. Shrinkage Baseline Advantage Estimator

```

1 prompt_mean = torch.mean(rewards, dim=1) # [n, m] => [n]
2 prompt_var = torch.var(rewards, dim=1, unbiased=True) / m
3
4 # Compute LOO batch mean and JS lambda
5 loo_means, js_lambdas = [], []
6 for i in range(n):
7     other = torch.cat([prompt_mean[:i], prompt_mean[i + 1:]])
8     batch_loo_mean = torch.mean(other)
9     v_square_i = torch.mean(torch.cat([prompt_var[:i], prompt_var[i + 1:]])
10    s_square_i = torch.mean((other - batch_loo_mean) ** 2)
11    js_lambda_i = v_square_i / (v_square_i + s_square_i)
12    js_lambda_i *= (n - 1) / n
13    loo_means.append(batch_loo_mean)
14    js_lambdas.append(js_lambda_i)
15
16 loo_means = torch.stack(loo_means) # [n]
17 js_lambdas = torch.stack(js_lambdas) # [n]
18 rloo_baseline = (torch.sum(rewards, dim=1, keepdim=True) - rewards) / (m - 1)
19 js_baseline = rloo_baseline * (1 - js_lambdas[:, None]) + loo_means[:, None] *
    ↪ js_lambdas[:, None]
20 advantage = rewards - js_baseline
    
```

D.2. Variance Recording

To track the gradient variance during training, we derive an unbiased estimator based on micro-batch gradients. Let i.i.d. micro-batch gradients $g_i \in \mathbb{R}^P$ for $i = 1, \dots, m$, with $\mu = \mathbb{E}[g_i]$ and $\Sigma = \text{Cov}(g_i)$. Let $g = \frac{1}{m} \sum_{i=1}^m g_i$ denote the batch gradient, and $\bar{g} = \frac{1}{m} \sum_{i=1}^m g_i$ the sample mean.

We first express the target quantity:

$$\text{Var}(g) = \text{tr}(\text{Cov}(g)) = \text{tr}(\text{Cov}(\frac{1}{m} \sum_{i=1}^m g_i)) = \text{tr}(\frac{1}{m^2} \sum_{i=1}^m \text{Cov}(g_i)) = \frac{1}{m} \text{tr}(\Sigma).$$

Using the identity $\sum_{i=1}^m \|g_i - \bar{g}\|^2 = \sum_{i=1}^m \|g_i\|^2 - m\|\bar{g}\|^2$, we take expectations term by term:

$$\mathbb{E}[\sum_{i=1}^m \|g_i - \bar{g}\|^2] = \sum_{i=1}^m \mathbb{E}\|g_i\|^2 - m \mathbb{E}\|\bar{g}\|^2.$$

For the two moments, we have

$$\mathbb{E}\|g_i\|^2 = \text{tr}(\Sigma) + \|\mu\|^2, \quad \mathbb{E}\|\bar{g}\|^2 = \mathbb{E}\|\frac{1}{m} \sum_{i=1}^m g_i\|^2 = \frac{1}{m} \text{tr}(\Sigma) + \|\mu\|^2.$$

Substituting back gives

$$\mathbb{E}[\sum_{i=1}^m \|g_i - \bar{g}\|^2] = m(\text{tr}(\Sigma) + \|\mu\|^2) - m(\frac{1}{m} \text{tr}(\Sigma) + \|\mu\|^2) = (m - 1) \text{tr}(\Sigma).$$

Dividing by $(m - 1)$ yields the unbiased sample-trace of Σ :

$$\mathbb{E}[\frac{1}{m-1} \sum_{i=1}^m \|g_i - \bar{g}\|^2] = \text{tr}(\Sigma).$$

Since $\text{Var}(g) = \frac{1}{m} \text{tr}(\Sigma)$, multiplying by $\frac{1}{m}$ produces the desired unbiased estimator of $\text{Var}(g)$:

$$\mathbb{E}[\frac{1}{m} \cdot \frac{1}{m-1} \sum_{i=1}^m \|g_i - \bar{g}\|^2] = \frac{1}{m} \text{tr}(\Sigma) = \text{Var}(g).$$

Finally, using $\sum_{i=1}^m \|g_i - \bar{g}\|^2 = \sum_{i=1}^m \|g_i\|^2 - \frac{1}{m} \|\sum_{i=1}^m g_i\|^2$, the estimator can be written in the form of:

$$\widehat{\text{Var}}(g) = \frac{1}{m} \cdot \frac{1}{m-1} (\sum_{i=1}^m \|g_i\|^2 - \frac{1}{m} \|\sum_{i=1}^m g_i\|^2).$$

Below is the pseudo code of gradient variance estimation:

Listing 2. Gradient Variance Estimation

```

1 sum_sq = 0.0 # accumulates sum_i ||g_i||^2
2 sum_g = zeros_like(vector) # accumulates sum_i g_i (shape: (P,))
3
4 for i in range(1, m + 1):
5     g_i = compute_flattened_gradient_for_microbatch(i) # shape: (P,)
6     sum_sq += dot(g_i, g_i) # scalar: ||g_i||^2
7     sum_g += g_i # vector: sum of g_i
8
9 # Sample-trace of covariance at micro-batch level
10 # (1/(m-1)) * sum_i ||g_i - g_bar||^2
11 trace_cov_micro = (sum_sq - dot(sum_g, sum_g) / m) / (m - 1)
12
13 # Unbiased estimate of batch-gradient variance trace
14 # Var(g) = tr(Cov(g)), because Cov(g) = (1/m) * Cov(g_i)
15 var_g_trace_estimate = (1.0 / m) * trace_cov_micro
    
```

D.3. Stability of the Estimated Shrinkage Coefficient

The shrinkage baseline relies on the plug-in estimate $\hat{\lambda}_i$ in Equation (15), computed from batch-level leave-one-out reward statistics. A natural concern is whether this estimate is stable enough in practice, particularly under the small batch sizes typical of our training setups. To check this directly, we record the variance of $\hat{\lambda}$ at every 100-step interval during training of Qwen2.5-1.5B-Instruct jointly on KnK-2 and KnK-3 with batch size 32. Table 5 reports the results.

Table 5. Variance of the estimated shrinkage coefficient $\hat{\lambda}$ during training of Qwen2.5-1.5B-Instruct on KnK-2 and KnK-3 with batch size 32. The variance is already small at step 100 and decreases by roughly two orders of magnitude as training proceeds.

Step	100	200	300	400	500	600	700	800	900	1000
$\text{Var}(\hat{\lambda})$	0.0057	0.0030	0.0018	0.0007	0.0002	0.0001	0.0002	0.0001	0.0002	0.0001

The variance of $\hat{\lambda}$ is already on the order of 10^{-3} at step 100 and drops below 10^{-3} from step 400 onward, suggesting that the per-batch shrinkage coefficient can be used directly without temporal smoothing or historical averaging.

D.4. Supplementary Results

We aggregate additional experimental results in this section.

D.4.1. COMPARISON WITH DIFFERENT BASELINES

We train the Qwen2.5-0.5B-Instruct model on the GSM8k dataset for 500 steps. GSM8k consists of 7,473 questions in the training set and 1,319 in the test set. For each rollout batch, we sample 64 distinct prompts, and for each prompt we generate $m \in \{2, 4, 8\}$ responses with the official template of GSM8k. Each experiment is repeated across five random seeds $\{0, 1, 2, 3, 4\}$. Summary of hyperparameters and configurations is provided in Section D.6. Detailed numbers are in Table 6.

D.4.2. COMPARISON WITH DAPO ON COUNTDOWN

Table 7 shows that JS provides consistent gains both on top of RLOO and on top of DAPO (Yu et al., 2025), indicating that our shrinkage baseline is orthogonal to DAPO’s algorithmic improvements.

D.4.3. KNIGHTS-AND-KNAVES SUBSET RESULTS

Table 6. Test Accuracy (%) across 5 runs for different algorithms under 2, 4, and 8 Generations. Each cell shows 5 accuracy values (%) at steps 100 to 500. Initial test accuracy is 40.03% for all.

Baseline	2 Generations	4 Generations	8 Generations
JS	48.29, 50.96, 53.56, 54.45, 54.68 47.55, 52.36, 53.77, 54.56, 56.12 50.11, 51.84, 52.06, 54.18, 55.27 49.10, 51.92, 54.12, 55.09, 54.97 48.58, 52.01, 53.66, 54.03, 55.07	50.22, 52.53, 56.56, 57.29, 58.03 50.51, 54.48, 56.56, 56.57, 57.80 50.83, 53.45, 55.41, 55.95, 56.92 51.43, 54.63, 55.85, 56.39, 56.29 52.37, 55.15, 56.59, 56.48, 57.59	52.19, 56.77, 57.65, 59.42, 60.12 53.28, 56.15, 56.53, 57.42, 57.89 53.39, 55.19, 56.92, 58.67, 58.73 52.77, 55.31, 58.13, 58.04, 58.24 52.51, 55.03, 57.03, 58.23, 59.68
BLOO	49.42, 50.80, 51.88, 52.23, 54.61 48.88, 51.43, 52.32, 53.39, 53.39 48.22, 50.42, 52.99, 54.41, 54.30 48.08, 49.95, 53.15, 53.43, 53.06 48.57, 51.46, 52.08, 53.98, 55.62	49.32, 52.65, 54.19, 55.16, 56.13 50.46, 53.39, 55.85, 55.11, 55.90 49.64, 52.43, 54.36, 54.47, 55.95 50.28, 52.86, 54.53, 54.51, 55.98 49.55, 52.53, 54.71, 54.91, 56.35	52.10, 54.61, 55.91, 56.91, 57.54 51.68, 54.59, 56.63, 56.07, 57.32 50.99, 54.25, 55.13, 55.68, 57.02 51.33, 54.71, 56.15, 55.16, 57.47 51.08, 54.98, 56.06, 56.95, 56.74
RLOO	49.23, 51.13, 52.02, 52.54, 54.89 48.48, 51.95, 52.84, 53.33, 53.56 47.95, 50.99, 53.37, 53.99, 54.63 47.98, 50.75, 51.81, 54.30, 54.91 47.46, 51.21, 51.64, 52.86, 54.45	51.01, 53.12, 55.62, 56.26, 56.10 50.84, 53.18, 54.24, 56.10, 55.28 50.42, 53.39, 54.53, 55.47, 56.60 50.10, 53.60, 54.59, 56.16, 56.92 51.19, 54.36, 56.29, 56.16, 56.80	52.99, 55.31, 56.89, 58.18, 58.50 52.92, 56.71, 56.71, 57.24, 58.01 52.25, 54.54, 56.29, 57.20, 58.18 52.62, 55.10, 56.03, 57.88, 58.63 52.34, 55.38, 56.86, 56.85, 58.21
GRPO	48.67, 51.99, 52.99, 54.72, 54.41 47.73, 52.42, 53.15, 54.06, 52.87 48.92, 51.08, 54.24, 54.12, 54.34 48.57, 51.72, 50.70, 52.89, 53.01 48.77, 51.52, 53.24, 53.49, 53.79	51.39, 53.54, 55.56, 55.94, 55.54 51.31, 52.96, 55.15, 55.79, 57.39 49.75, 53.51, 54.38, 54.57, 55.21 50.05, 54.43, 56.62, 57.77, 56.88 51.63, 53.04, 54.47, 55.09, 56.19	51.96, 56.07, 57.10, 58.27, 58.20 52.52, 55.53, 56.51, 57.80, 59.06 52.43, 55.57, 57.12, 58.37, 59.05 52.43, 55.77, 57.56, 57.54, 57.21 52.48, 55.31, 57.83, 57.85, 57.91
ReMax	49.16, 52.12, 52.40, 52.54, 54.83 48.76, 52.68, 53.37, 54.39, 54.10 48.79, 52.10, 52.45, 53.75, 55.65 48.57, 51.52, 52.98, 52.40, 54.06 48.92, 51.63, 52.54, 53.33, 54.86	50.05, 53.37, 55.35, 55.95, 56.33 49.08, 51.15, 53.92, 55.69, 55.94 50.19, 52.83, 54.98, 54.59, 57.12 48.78, 51.31, 54.12, 54.98, 56.57 49.48, 53.80, 54.16, 55.25, 56.40	51.04, 54.71, 56.65, 57.10, 57.81 52.01, 55.71, 56.29, 57.10, 59.38 51.55, 54.22, 55.66, 57.26, 57.62 51.05, 54.39, 55.69, 55.36, 56.19 51.32, 54.84, 55.99, 56.87, 57.81
REINFORCE++	48.86, 51.90, 53.80, 53.57, 55.01 49.61, 52.18, 53.06, 54.86, 55.27 48.19, 50.54, 52.66, 53.74, 53.51 49.51, 51.96, 52.27, 54.63, 55.33 48.76, 51.35, 52.46, 53.98, 54.96	50.05, 52.87, 53.77, 55.44, 56.79 49.95, 53.25, 54.98, 54.97, 53.69 49.11, 53.33, 55.19, 55.72, 56.04 49.63, 52.24, 53.72, 55.13, 54.01 49.52, 52.68, 54.86, 55.21, 55.82	51.51, 54.50, 55.59, 58.10, 57.92 51.17, 54.57, 57.22, 57.15, 57.24 52.08, 55.82, 57.16, 57.13, 57.00 51.95, 54.84, 57.03, 57.77, 57.04 50.99, 53.82, 56.94, 57.23, 57.32

Table 7. Test score (%) on the Countdown benchmark. JS provides consistent gains when applied on top of either RLOO or DAPO, indicating that the shrinkage baseline is orthogonal to the algorithmic improvements introduced by DAPO.

Method	Countdown (%)
GRPO	53.2
RLOO	51.7
RLOO + JS	60.9 (+9.2)
DAPO	59.2
DAPO + JS	65.5 (+6.3)

Table 8. Detailed test accuracy (%) on different subsets for Knights-and-Knaves experiments. A larger number in KnK indicates a more complex task.

Algorithm	Train on KnK-Easy			Train on KnK-Hard		
	KnK-4	KnK-5	KnK-6	KnK-6	KnK-7	KnK-8
RLOO	58.38	49.89	41.25	39.50	30.00	16.75
JS	75.87	66.16	55.25	42.13	32.50	25.38

D.4.4. MULTI-SEED ROBUSTNESS

We repeat the Countdown experiment on Qwen2.5-3B and the Knights-and-Knaves (KnK) experiments on Qwen2.5-1.5B-Instruct across three random seeds. For every reported step, JS yields a higher accuracy than RLOO in all three seeds, indicating that the shrinkage baseline provides a consistent improvement.

Table 9. Multi-seed validation accuracy (%) on Countdown for Qwen2.5-3B. Each row reports a single seed; the bottom row of each block reports mean $\pm$ std across three seeds. JS outperforms RLOO at every reported step in every seed.

Method	120	140	160	180	200
RLOO (Run 1)	52.2	52.5	52.0	52.4	52.5
RLOO (Run 2)	51.4	51.7	51.6	52.9	53.2
RLOO (Run 3)	54.3	55.0	54.9	57.5	58.4
RLOO mean $\pm$ std	52.6 $\pm$ 1.2	53.1 $\pm$ 1.4	52.8 $\pm$ 1.5	54.3 $\pm$ 2.3	54.7 $\pm$ 2.6
JS (Run 1)	54.6	56.4	57.4	59.2	60.9
JS (Run 2)	58.2	59.4	61.3	64.1	64.0
JS (Run 3)	61.7	63.6	64.3	64.6	65.5
JS mean $\pm$ std	58.2 $\pm$ 2.9	59.8 $\pm$ 3.0	61.0 $\pm$ 2.8	62.6 $\pm$ 2.4	63.5 $\pm$ 1.9

Table 10. Multi-seed validation accuracy (%) on KnK-2 for Qwen2.5-1.5B-Instruct trained jointly on KnK-2 and KnK-3. Each row reports a single seed; the bottom row of each block reports mean $\pm$ std across three seeds.

Method	600	700	800	900	1000
RLOO (Run 1)	79.5	83.7	87.0	87.0	86.6
RLOO (Run 2)	71.5	75.0	79.6	81.7	85.0
RLOO (Run 3)	79.6	79.6	84.6	81.2	83.6
RLOO mean $\pm$ std	76.9 $\pm$ 3.8	79.5 $\pm$ 3.6	83.7 $\pm$ 3.1	83.3 $\pm$ 2.6	85.1 $\pm$ 1.2
JS (Run 1)	86.4	84.6	84.4	86.1	89.2
JS (Run 2)	75.6	77.6	83.2	86.6	86.8
JS (Run 3)	86.0	86.4	88.5	87.7	89.4
JS mean $\pm$ std	82.7 $\pm$ 5.0	82.9 $\pm$ 3.8	85.4 $\pm$ 2.3	86.8 $\pm$ 0.7	88.5 $\pm$ 1.2

Table 11. Multi-seed validation accuracy (%) on KnK-3 for Qwen2.5-1.5B-Instruct trained jointly on KnK-2 and KnK-3. Each row reports a single seed; the bottom row of each block reports mean $\pm$ std across three seeds.

Method	600	700	800	900	1000
RLOO (Run 1)	47.0	53.1	57.9	58.4	57.9
RLOO (Run 2)	48.2	45.8	49.9	48.4	51.9
RLOO (Run 3)	48.6	49.4	50.9	48.2	55.6
RLOO mean $\pm$ std	48.0 $\pm$ 0.7	49.4 $\pm$ 3.0	52.9 $\pm$ 3.6	51.7 $\pm$ 4.7	55.1 $\pm$ 2.5
JS (Run 1)	59.6	62.4	66.9	65.9	62.6
JS (Run 2)	49.4	48.8	52.2	53.6	50.7
JS (Run 3)	59.4	61.3	62.1	62.5	60.8
JS mean $\pm$ std	56.1 $\pm$ 4.8	57.5 $\pm$ 6.2	60.4 $\pm$ 6.1	60.7 $\pm$ 5.2	58.0 $\pm$ 5.2

D.4.5. ADDITIONAL TRAINING CURVES

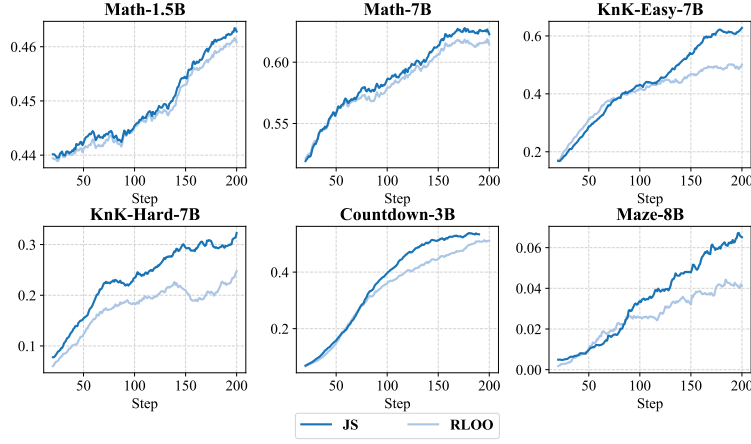


Figure 13. Moving average reward curves during training for the shrinkage baseline and the RLOO baseline.

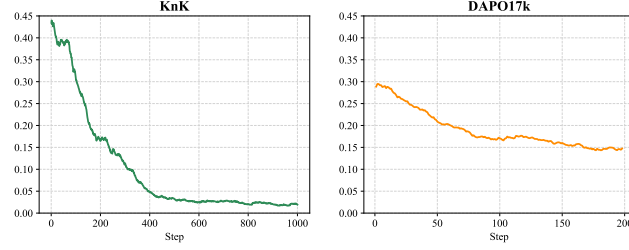


Figure 14. The moving average shrinkage coefficient curves on different datasets. Left: KnK dataset; Right: DAP017k dataset. Adaptive shrinkage enables dynamically optimal baseline estimation that adjusts to the data distribution and training progress.

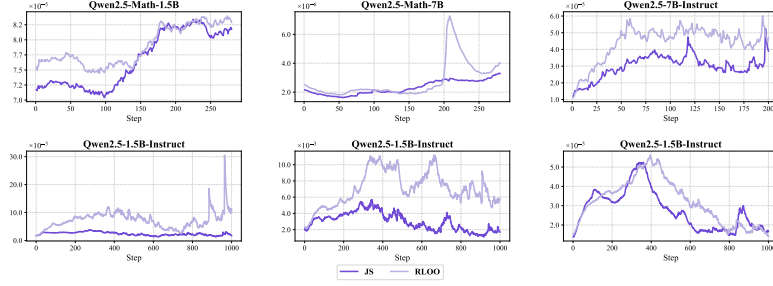


Figure 15. Additional running average gradient variance curves during training.

D.5. Task Description

We describe the tasks used in our experiments in this section.

D.5.1. MATH REASONING

For math reasoning, we train Qwen2.5-Math-1.5B (Team, 2024) and Qwen2.5-Math-7B (Team, 2024) models on DAPO17k (Yu et al., 2025) and Math12k (Hendrycks et al., 2021) datasets, and Qwen3-4B-Base (Yang et al., 2025) on DAPO17k dataset. An example of the prompt template and its correct response is shown below.

Math Reasoning Example (Qwen2.5-Math-7B)

```
<|im_start|>system
Please reason step by step and put the final answer in \boxed{ }. <|im_end|>
<|im_start|>user
Cities A and B are 45 miles apart. Alicia lives in A and Beth lives in B. Alicia bikes
towards B at 18 miles per hour. Leaving at the same time, Beth bikes toward A at 12
miles per hour. How many miles from City A will they be when they meet? Let's think
step by step and output the final answer within \boxed{ }. <|im_end|>
<|im_start|>assistant
To determine where Alicia and Beth will meet, we need to find out how long it will take
for them to meet and then use that time to calculate the distance Alicia has traveled.
1. First, let's calculate their combined speed. Since Alicia bikes at 18 miles per
hour and Beth bikes at 12 miles per hour, their combined speed is  $18 + 12 = 30$  miles
per hour.
2. Since the distance between the two cities is 45 miles, the time it takes for them
to meet is the total distance divided by their combined speed. So, the time  $t$  it takes
for them to meet is  $t = 45/30 = 1.5$  hours.
3. Now, we need to find out how far Alicia has traveled in 1.5 hours. Since Alicia
bikes at 18 miles per hour, the distance she travels in 1.5 hours is  $18 * 1.5 = 27$ 
miles.
So, Alicia and Beth will meet 27 miles from City A. The final answer is \boxed{27}
```

D.5.2. LOGIC PUZZLE REASONING

We experiment on three representative logic puzzle tasks: Knights-and-Knaves, Countdown, and Maze.

Knights-and-Knaves (KnK) is a classic logic puzzle where the goal is to determine truth-tellers (knights) and liars (knaves) based on their statements. We train on two datasets: KnK-Easy, which consists of 10k problems each with 4, 5, or 6 people, and KnK-Hard, which consists of 10k problems each with 6, 7, or 8 people. For each experiment, we evaluate on 300 questions sampled from the same types of KnK puzzles used in training.

Countdown is a numerical game where players use arithmetic operations on given numbers to reach a target value. We train on 10k questions with three numbers and 10k questions with four numbers, and evaluate on 200 questions from the same distribution.

Maze is a spatial puzzle that requires navigating through a complex grid of walls and paths to reach the finish point. We train on 20k 6x6 puzzles where the correct path length ranges from 4 to 8, and evaluate on 200 puzzles from the same distribution.

We provide examples of question content and their corresponding correct responses for each puzzle below.

Knights-and-Knaves

Question:

A very special island is inhabited only by sages and fools. Sages always tell the truth, and fools always lie. You meet 6 inhabitants: Owen, Liam, Evelyn, William, Sofia, and Emily. As Owen put it, "William is a sage". Liam was heard saying, "William is a fool". Evelyn noted, "Sofia is a fool and Evelyn is a sage". "if Owen is a sage then Sofia is a sage," William mentioned. "if Liam is a sage then Evelyn is a sage," Sofia claimed. Emily told you that if Emily is a sage then Liam is a fool. So who is a sage and who is a fool? Let's think step by step and output the final answer within `\boxed{}`.

Response:

We need to consider two cases for Owen: (1) If Owen is a sage, then his statement is true, so William must be a sage. (2) If Owen is a fool, then his statement is false, so William must be a fool.

Liam's statement: "William is a fool." If Liam is a sage, William is a fool. If Liam is a fool, William is a sage.

William's statement: "If Owen is a sage then Sofia is a sage." Since Sofia is a sage, William's statement is true. Therefore, William must be a sage.

Owen's statement revisited: "William is a sage." Since William is a sage, Owen's statement is true. Therefore, Owen must be a sage.

Liam's statement revisited: Since William is a sage, Liam's statement is false. Therefore, Liam must be a fool.

Emily's statement: "If Emily is a sage then Liam is a fool." Since Emily is a sage, Liam must be a fool. This confirms our previous determination.

Therefore: Owen is a sage, Liam is a fool, Evelyn is a fool, William is a sage, Sofia is a sage, and Emily is a sage.

`\boxed{Owen is a sage, Liam is a fool, Evelyn is a fool, William is a sage, Sofia is a sage, and Emily is a sage}`

Countdown

Question:

Using the numbers [55, 53, 59, 22], create an equation that equals 71. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in `<think>` `</think>` tags. And return the final answer in `<answer>` `</answer>` tags, for example `<answer> (1 + 2) / 3 </answer>`.

Response:

`<think>`

We have the numbers 55, 53, 59, and 22. We need an equation that uses each at most once to make 71. Try combinations:

55-53-59+22=-45 (not it)

55-53+59-22=39 (not it)

55+53-59-22=27 (not it)

59+22-55-53=-27 (not it)

59-22+53-55=35 (not it)

53+22+55-59=71 (works!)

Therefore, an expression that equals 71 is 53+22+55-59.

`</think>`

`<answer> 53+22+55-59 </answer>`

Maze

Question:

You need to solve the following maze. * denotes the wall that you cannot walk through, . denotes available area that you can walk through. S denotes the starting point, E denotes the destination. There are four movement actions: Left (L), Right (R), Up (U), Down (D). Write the final path in <answer> </answer>.

Maze

..E

...

**.*.*

**S.*

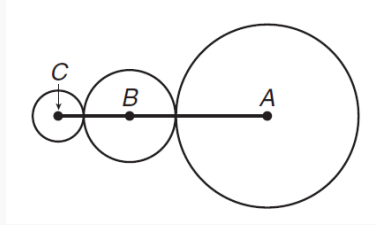
Response:

1. Locate the start and end: Start S at (5,4), End E at (2,5).
 2. Check immediate moves from S(5,4): U blocked, D blocked, L open, R open.
 3. Choose a route to approach E:
 - Move R: (5,4)→(5,5), open.
 - Move U: (5,5)→(4,5), open.
 - Move U: (4,5)→(3,5), open.
 - Move U: (3,5)→(2,5), destination reached.
 4. Validate: Each step lands on open cell until reaching E. No wall crossed.
- <answer> RUUU </answer>

D.5.3. MULTIMODAL REASONING

We evaluate on Geometry3k (Lu et al., 2021), a geometry problem dataset that requires visual understanding and mathematical reasoning. Models must interpret figures and solve problems involving shapes, angles, and spatial relationships. Below is an example from the dataset.

Geometry3k

Question:


In the figure, the radius of circle A is twice the radius of circle B and four times the radius of circle C. If the sum of the circumferences of the three circles is 42π , find the measure of $\overline{AC}$.

Response:

Let the radius of circle C be r . Then the radius of circle B is $2r$ and the radius of circle A is $4r$.

The circumference of a circle is 2π times its radius. So the sum of the circumferences is: $2\pi r + 2\pi(2r) + 2\pi(4r) = 14\pi r$.

Given that the sum equals 42π : $14\pi r = 42\pi$, so $r = 3$.

Since the circles are externally tangent, $\overline{AC}$ equals the sum of the radii of all three circles: $\overline{AC} = r + 2r + 2r + 4r = 9r = 27$.

The answer is $\boxed{27}$.

D.6. Experimental Setups

We summarize the training configurations for all experiments in this section.

Training setup of Qwen2.5-Math-1.5B

Parameter	Value	Parameter	Value
Base model	Qwen2.5-Math-1.5B	Training set	DAPO17k, Math12k
Test set	MATH500, AMC23, Olympiad	Prompts per batch	64
Generations per prompt	4	Grad updates per step	2
Max prompt len	1024	Max response len	2048
Learning rate	2×10^{-6}	Clip ratio	0.22
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	0.8	Validation temp	0.8
Validation samples	16	Validation interval	50 steps
LoRA Rank	0	Device	4×GH200

Training setup of Qwen2.5-Math-7B

Parameter	Value	Parameter	Value
Base model	Qwen2.5-Math-7B	Training set	DAPO17k, Math12k
Test set	MATH500, AMC23, Olympiad	Prompts per batch	64
Generations per prompt	4	Grad updates per step	2
Max prompt len	1024	Max response len	2048
Learning rate	2×10^{-5}	Clip ratio	0.22
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	0.8	Validation temp	0.8
Validation samples	16	Validation interval	50 steps
LoRA Rank	256	Device	4×GH200

Training setup of Qwen3-4B-Base

Parameter	Value	Parameter	Value
Base model	Qwen3-4B-Base	Training set	DAPO17k
Test set	MATH500, AMC23, Olympiad	Prompts per batch	64
Generations per prompt	4	Grad updates per step	2
Max prompt len	1024	Max response len	3072
Learning rate	2×10^{-6}	Clip ratio	0.28
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	1.0	Validation temp	1.0
Validation samples	16	Validation interval	50 steps
LoRA Rank	0	Device	4×H100

Training setup for KnK-Easy

Parameter	Value	Parameter	Value
Base model	Qwen2.5-7B-Instruct	Training set	KnK-4, KnK-5, KnK-6
Test set	KnK-4/5/6-Test	Prompts per batch	32
Generations per prompt	8	Grad updates per step	2
Max prompt len	1024	Max response len	2048
Learning rate	4×10^{-5}	Clip ratio	0.22
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	0.7	Validation temp	0.7
Validation samples	16	Validation interval	25 steps
LoRA Rank	256	Device	4×GH200

Training setup for KnK-Hard

Parameter	Value	Parameter	Value
Base model	Qwen2.5-7B-Instruct	Training set	KnK-6, KnK-7, KnK-8
Test set	KnK-6/7/8-Test	Prompts per batch	32
Generations per prompt	8	Grad updates per step	2
Max prompt len	1024	Max response len	2048
Learning rate	4×10^{-5}	Clip ratio	0.22
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	0.7	Validation temp	0.7
Validation samples	16	Validation interval	25 steps
LoRA Rank	256	Device	4×GH200

Training setup for Countdown

Parameter	Value	Parameter	Value
Base model	Qwen2.5-3B	Training set	Countdown3, Countdown4
Test set	Countdown3/4-Test	Prompts per batch	64
Generations per prompt	5	Grad updates per step	2
Max prompt len	512	Max response len	1024
Learning rate	1×10^{-6}	Clip ratio	0.22
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	0.7	Validation temp	0.7
Validation samples	16	Validation interval	10 steps
LoRA Rank	0	Device	2×GH200

Training setup for Maze

Parameter	Value	Parameter	Value
Base model	Ministral-8B-Instruct	Training set	Maze6x6
Test set	Maze6x6-Test	Prompts per batch	32
Generations per prompt	8	Grad updates per step	2
Max prompt len	1024	Max response len	2048
Learning rate	3×10^{-7}	Clip ratio	0.25
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	0.7	Validation temp	0.7
Validation samples	16	Validation interval	25 steps
LoRA Rank	0	Device	4×GH200

Training Setup for GSM8k

Parameter	Value	Parameter	Value
Base model	Qwen2.5-0.5B-Instruct	Training set	GSM8k-Train
Test set	GSM8k-Test	Prompts per batch	64
Generations per prompt	4	Grad updates per step	1
Max prompt len	1024	Max response len	2048
Learning rate	1×10^{-6}	Clip ratio	0.2
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	0.7	Validation temp	0.5
Validation samples	10	Validation interval	100 steps
		Device	1×GH200

Training setup of Qwen2.5-VL-3B-Instruct

Parameter	Value	Parameter	Value
Base model	Qwen2.5-VL-3B-Instruct	Training set	Geo3k
Test set	Geo3k-Test	Prompts per batch	512
Generations per prompt	5	Grad updates per step	4
Max prompt len	1024	Max response len	1024
Learning rate	1×10^{-6}	Clip ratio	0.2
KL coeff	0.01	Entropy coeff	0.0
Rollout temp	0.7	Validation temp	0.7
Validation samples	16	Validation interval	20 steps
LoRA Rank	0	Device	4×GH200

Training setup of Qwen2.5-VL-7B-Instruct

Parameter	Value	Parameter	Value
Base model	Qwen2.5-VL-7B-Instruct	Training set	Geo3k
Test set	Geo3k-Test	Prompts per batch	512
Generations per prompt	5	Grad updates per step	4
Max prompt len	1024	Max response len	1024
Learning rate	1×10^{-6}	Clip ratio	0.2
KL coeff	0.01	Entropy coeff	0.0
Rollout temp	0.7	Validation temp	0.7
Validation samples	16	Validation interval	20 steps
LoRA Rank	0	Device	4×GH200

Training setup of Llama-3.2-3B-Instruct

Parameter	Value	Parameter	Value
Base model	Llama-3.2-3B-Instruct	Reward model	Llama-3-8B-RM-700k
Training set	prompt-collection-v0.1 (179k)	Test set	Arena-Hard, AC-Writing
Prompts per batch	512	Generations per prompt	2
Grad updates per step	8	Epochs	1
Max prompt len	1024	Max response len	1024
Learning rate	5×10^{-7}	Clip ratio	0.2
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	1.0	LoRA Rank	0
Device	8×H100		

Training setup of Qwen2.5-1.5B-Instruct

Parameter	Value	Parameter	Value
Base model	Qwen2.5-1.5B-Instruct	Training set	KnK-2, KnK-3
Test set	KnK-2/3-Test	Prompts per batch	32
Generations per prompt	8	Grad updates per step	2
Max prompt len	1024	Max response len	2048
Learning rate	5×10^{-7}	Clip ratio	0.22
KL coeff	0.0	Entropy coeff	0.0
Rollout temp	0.7	Validation temp	0.7
Validation samples	16	Validation interval	25 steps
LoRA Rank	0	Device	4×H100